

Curso MLOps / DataOps

Manual completo del alumno · ANBAN

José Picón

2026

La hoja de ruta del curso, en 8 comandos

Los 8 comandos

Qué hace cada lab

Tres URLs que deben funcionar tras `./setup.sh`

Reglas de oro para no atascarse

Dónde buscar más detalle de cada paso

Antes de empezar: qué vas a instalar y por qué

A · Windows 10 / Windows 11

A.1 Instalar WSL (Windows Subsystem for Linux)

A.2 Instalar Docker Desktop

A.3 Si no quieres usar WSL: PowerShell nativo

B · macOS (Intel y Apple Silicon)

B.1 Identifica tu chip

B.2 Instalar Docker Desktop

B.3 Primer arranque

B.4 Verificar

C · Linux

C.1 Ubuntu / Debian

C.2 Fedora

C.3 Arch Linux

C.4 Verificar

D · Comprobación final (todos los sistemas)

E · Descargar y descomprimir el material del curso

Dónde descomprimirlo

Cómo descomprimir, paso a paso

F · Levantar el laboratorio del curso

Qué hace `./setup.sh` (o `setup.ps1`)

Lo que debes ver al final

Comprobar que los tres servicios están vivos

G · Si algo falla: el doctor automático

H · Problemas frecuentes y cómo solucionarlos

Docker no arranca o la ballena queda animada eternamente

“Cannot connect to the Docker daemon”

Una imagen no se descarga (timeout, 500 Internal Server Error)

Puerto ya ocupado por otro proceso

El laboratorio funciona pero un alumno tiene una pantalla negra al abrir Jupyter

Ejercicio para fijar lo aprendido

1. La pregunta con la que empieza todo

2. Cuatro disciplinas que se confunden

3. El triángulo Código-Datos-Modelo

4. Ciclo de vida: CRISP-DM y CRISP-ML(Q)

5. Niveles de madurez MLOps (modelo de Google)

5.1 Nivel 0 — Proceso manual

5.2 Nivel 1 — Pipeline ML automatizado

5.3 Nivel 2 — CI/CD completo

6. Roles en un equipo MLOps

7. Ejercicio: diagnóstico de madurez

Escenario A

Escenario B

Escenario C

Solución sugerida

8. Vocabulario mínimo

9. Para profundizar

1. Qué es DataOps y por qué te importa

2. La herramienta principal: DVC

2.1 Qué es DVC

2.2 Por qué no basta con Git para datos

2.3 Alternativas a DVC

2.4 Instalación de DVC

3. La herramienta secundaria: Great Expectations (y su alternativa Pandera)

3.1 Qué es Great Expectations

3.2 Por qué no usamos Great Expectations directamente

3.3 La alternativa ligera: Pandera

4. Las cinco dimensiones de calidad del dato

5. Antes de la práctica: requisitos

6. Ejercicio: montar un pipeline reproducible con DVC

6.1 Cómo hacer este ejercicio

6.2 Paso 1 — Inicializar Git

6.3 Paso 2 — Inicializar DVC

6.4 Paso 3 — Configurar el remoto MinIO

6.5 Paso 4 — Trackear el dataset

6.6 Paso 5 — Escribir las expectations

6.7 Paso 6 — Crear el pipeline

6.8 Paso 7 — Ver la caché por dependencias en acción

6.9 Paso 8 — Romper el dato a propósito

6.10 Paso 9 — Recuperar el dato bueno

6.11 Paso 10 — Simular un compañero nuevo

7. Comandos de DVC explicados uno por uno

8. Pandas y scikit-learn: las librerías de soporte

8.1 Pandas

8.2 Scikit-learn

9. Checklist de cierre

10. Para profundizar
 1. El problema que resuelve el tracking de experimentos
 2. La herramienta principal: MLflow
 - 2.1 Qué es MLflow
 - 2.2 Vocabulario clave
 - 2.3 Alternativas a MLflow
 - 2.4 Instalación de MLflow
 3. Las librerías auxiliares de este módulo
 - 3.1 Scikit-learn (modelo “logreg” y “rf”)
 - 3.2 XGBoost (modelo “xgb”)
 - 3.3 Boto3 (cliente S3 para MinIO)
 4. Antes de la práctica: requisitos
 5. Ejercicio: trackear 3 modelos y registrar el mejor
 - 5.1 Paso 1 — Exportar las variables de entorno
 - 5.2 Paso 2 — Repasar el script de entrenamiento
 - 5.3 Paso 3 — Entrenar los tres modelos
 - 5.4 Paso 4 — Comparar los runs en la UI
 - 5.5 Paso 5 — Registrar el modelo ganador
 - 5.6 Paso 6 — Promocionar a Staging
 - 5.7 Paso 7 — Cargar el modelo desde Python
 6. Funciones de MLflow explicadas una a una
 7. Métricas de clasificación explicadas
 8. Checklist de cierre
9. Para profundizar
 1. Servir un modelo: qué significa y qué patrones existen
 2. Las librerías principales: FastAPI, Pydantic, Uvicorn
 - 2.1 Qué es FastAPI
 - 2.2 Qué es Pydantic
 - 2.3 Qué es Uvicorn
 - 2.4 Instalación de las tres
 3. La otra parte: Docker, en serio
 - 3.1 Imagen vs contenedor
 - 3.2 El Dockerfile

- 3.3 Construir la imagen
- 3.4 Lanzar un contenedor
- 3.5 Comandos útiles para gestionar contenedores
- 4. La aplicación: lectura del main.py
 - 4.1 Imports y configuración
 - 4.2 Estado global y schema de entrada
 - 4.3 Lifespan: cargar el modelo al arrancar
 - 4.4 Endpoints
- 5. Antes de la práctica: requisitos
- 6. Ejercicio: empaquetar y servir el modelo
 - 6.1 Paso 1 — Construir la imagen
 - 6.2 Paso 2 — Lanzar el contenedor
 - 6.3 Paso 3 — Probar /health
 - 6.4 Paso 4 — Probar /version
 - 6.5 Paso 5 — Hacer una predicción real
 - 6.6 Paso 6 — Probar la validación con un dato malo
 - 6.7 Paso 7 — Probar el endpoint de métricas
 - 6.8 Paso 8 — Probar la documentación automática
 - 6.9 Paso 9 — Ver los logs
 - 6.10 Paso 10 — Detener el contenedor
- 7. Conceptos de performance
- 8. Checklist de cierre
- 9. Para profundizar
 - 1. Las tres siglas: CI, CD y CT
 - 2. Estrategias de despliegue
 - 3. Monitorización: dos familias de métricas
 - 3.1 Métricas de servicio
 - 3.2 Métricas de modelo
 - 4. Drift: la enfermedad silenciosa
 - 4.1 Tests estadísticos para detectar drift
 - 4.2 PSI explicado a fondo
 - 5. Las librerías principales
 - 5.1 Evidently

5.2 Numpy

5.3 GitHub Actions

6. Antes de la práctica: requisitos

7. Ejercicio: detectar drift y promoción condicional

7.1 Paso 1 — Exportar variables de entorno

7.2 Paso 2 — Generar un dataset con drift sintético

7.3 Paso 3 — Generar el reporte de drift

7.4 Paso 4 — Interpretar los resultados

7.5 Paso 5 — La lógica de promoción condicional

7.6 Paso 6 — Probarlo en local

7.7 Paso 7 — Revisar el workflow de GitHub Actions

8. Checklist de cierre

9. Para profundizar

1. Más allá del MVP: gobernanza del modelo

2. La Model Card

2.1 Qué debe contener

2.2 Generarla automáticamente

3. Audit trail

4. RGPD y AI Act

4.1 RGPD (Reglamento General de Protección de Datos)

4.2 AI Act europeo

5. Sesgo y fairness

5.1 Decisión humana

6. Reproducibilidad estricta

7. Costes y FinOps en ML

8. LLMOps en un vistazo

9. Antes de la práctica: requisitos

10. Ejercicio final: recorrido end-to-end + Model Card

10.1 Paso 1 — Comprobar que todas las piezas están vivas

10.2 Paso 2 — Verificar la cadena de artefactos

10.3 Paso 3 — Generar la Model Card automáticamente

10.4 Paso 4 — Calcular métricas por subgrupo (ejercicio práctico)

10.5 Paso 5 — Rellenar las secciones manuales

10.6 Paso 6 — Checklist final del curso

11. Hoja de ruta de aprendizaje

12. Bibliografía y recursos

Libros

Webs

Whitepapers fundacionales

Stack del curso

URLs y credenciales

Variables de entorno (ponlas en cada terminal nueva)

DVC

Git + DVC

MLflow

Desde Python

Model Registry

FastAPI + Docker

Probar la API con curl

Locust (stress test)

Evidently y drift

Promote condicional

Atajos para depuración

Si algo se rompe sin remedio

La hoja de ruta del curso, en 8 comandos

Esta es la única hoja que necesitas tener a mano. Imprime esta página si quieres. Todo lo que vas a teclear durante los dos días está aquí.

Los 8 comandos

```
# (1) Sólo la primera vez del día: arrancar el laboratorio
./setup.sh

# (2) DÍA 1 · mañana – DataOps con DVC
cd labs/lab1_dataops && ./run.sh

# (3) DÍA 1 · tarde – Tracking con MLflow
cd ../lab2_mlflow_dvc && ./run.sh

# (4) DÍA 2 · mañana – Servir el modelo (FastAPI + Docker)
cd ../lab3_serving && ./run.sh

# (5) DÍA 2 · tarde – CI/CD y detección de drift
cd ../lab4_cicd_monitoring && ./run.sh

# (6) CIERRE – caso end-to-end y Model Card
cd ../lab5_e2e && ./run.sh

# (7) Al terminar la sesión, parar Docker
cd ../../ && ./shutdown.sh

# (8) Al día siguiente, volver a empezar
./setup.sh
```

Si **algo se rompe** en cualquier momento:

```
./doctor.sh
```

En Windows nativo (sin WSL), reemplaza `./setup.sh` por `.\setup.ps1` y `./run.sh` por `.\run.ps1`. En todo lo demás, igual.

Qué hace cada lab

Cada lab tiene un script `run.sh` (o `run.ps1` en Windows nativo) que te va guiando paso a paso. Tú solo pulsas Enter y lees lo que sale en pantalla.

Lab	Tiempo	Qué aprendes
Lab 1 — DataOps con DVC	40-50 min	Versionar datasets, pipeline reproducible, detectar datos rotos
Lab 2 — MLflow Tracking	50 min	Entrenar 3 modelos, comparar y promocionar el mejor
Lab 3 — Serving	40 min	Empaquetar el modelo como API REST con Docker
Lab 4 — CI/CD + Drift	50 min	Promoción condicional y detección de drift en producción
Lab 5 — End-to-end	45 min	Recorrido completo y generación de Model Card

Tres URLs que deben funcionar tras `./setup.sh`

Si abres estos tres enlaces en el navegador y los tres responden, estás dentro del curso:

Servicio	URL	Credenciales
Jupyter	<code>http://localhost:8888</code>	token: <code>anban</code>
MLflow	<code>http://localhost:5050</code>	—
MinIO (consola)	<code>http://localhost:9001</code>	<code>minio</code> / <code>minio12345</code>

Si alguno no responde:

1. Espera 30 segundos y prueba de nuevo (a veces tarda en arrancar).
2. Si sigue sin ir, ejecuta `./doctor.sh`.
3. Comparte la salida con tu profesor si no entiendes el mensaje.

Reglas de oro para no atascarse

1. **No te saltes el orden.** Cada lab depende del anterior. Lab 2 necesita los parquet del Lab 1. Lab 3 necesita el modelo del Lab 2. Etc.
2. **Lee lo que sale en pantalla.** No es ruido: cada bloque explica qué está pasando y por qué importa.
3. **Si te atascas,** `./doctor.sh`. Está hecho exactamente para esto.
4. **Al terminar la sesión,** `./shutdown.sh`. Si dejas Docker abierto sin necesidad, te come batería y RAM.

Dónde buscar más detalle de cada paso

Si quieres entender la teoría o el porqué de cada comando que ejecutas con `./run.sh` :

- **Lab 1:** `material_alumno/02_dataops_dvc_ge.pdf`
- **Lab 2:** `material_alumno/03_mlflow_tracking.pdf`
- **Lab 3:** `material_alumno/04_serving_fastapi.pdf`
- **Lab 4:** `material_alumno/05_cicd_drift.pdf`
- **Lab 5:** `material_alumno/06_governance_e2e.pdf`

Y para la chuleta de comandos, `material_alumno/cheatsheet_comandos.pdf`.

Antes de empezar: qué vas a instalar y por qué

Este capítulo te lleva desde “ordenador recién encendido” hasta “todo el laboratorio del curso levantado”. Cada paso va explicado: qué hace, qué tienes que ver en pantalla, qué hacer si algo no sale como esperas.

Cuando termines este capítulo, tendrás:

- **Docker Desktop** (Windows y macOS) o **Docker Engine** (Linux) instalado y funcionando. Es lo que permite ejecutar contenedores, que son cajas autocontenidas con software listo para usar.
- **Una terminal** donde teclear los comandos del curso. En Linux y macOS viene de serie. En Windows hay dos opciones: PowerShell (nativo) o WSL (un Linux dentro de Windows).
- **Cinco contenedores arrancados** que forman el laboratorio: Jupyter, MLflow, MinIO, PostgreSQL y un inicializador de buckets.

No necesitas instalar Python, ni Git, ni librerías sueltas en el sistema. Todo eso va dentro de los contenedores. Eso es la gracia de Docker: una vez instalado, todo lo demás “ya viene”.

Tiempo estimado de instalación completa: 30 minutos la primera vez. La segunda, 2 minutos (solo levantar lo que ya descargaste).

A • Windows 10 / Windows 11

En Windows tienes dos caminos. Lee los dos antes de decidir.

Opción	Cuándo elegirla	Ventajas	Inconvenientes
WSL (Ubuntu dentro de Windows)	Recomendado. Si no tienes restricción de tu empresa.	Es Linux real, rendimiento óptimo de Docker. Los comandos del curso funcionan tal cual.	Hay que instalar WSL (10 minutos) la primera vez.
PowerShell nativo	Si no puedes/no quieres instalar WSL	No instalas nada extra de sistema.	Algunos comandos del curso requieren adaptación. Docker más lento con volúmenes.

Si dudas, escoge **WSL**. El resto de esta sección lo explica.

A.1 Instalar WSL (Windows Subsystem for Linux)

WSL es Linux ejecutándose dentro de Windows, sin emulación. Tu ordenador no se reinicia en otro sistema; conviven los dos.

Paso 1.1 — Comprobar que tienes Windows 10 (build 19041+) o Windows 11

Pulsa la tecla Windows + R. En la ventana “Ejecutar”, escribe:

```
winver
```

Pulsa Enter. Se abre una ventana con tu versión. Lee la línea “Versión 19041” (o superior) o “Versión 22H2”, etc. Si la versión es inferior a 19041, primero actualiza Windows (Inicio → Configuración → Windows Update → Buscar actualizaciones).

Paso 1.2 — Abrir PowerShell como administrador

Pulsa la tecla Windows. Escribe:

```
powershell
```

En los resultados, sobre “Windows PowerShell”, **botón derecho** → **Ejecutar como administrador**. Windows te pedirá permiso, di que sí. Aparece una ventana azul oscuro con texto blanco.

Paso 1.3 — Lanzar la instalación de WSL

En esa ventana de PowerShell teclea exactamente:

```
wsl --install -d Ubuntu
```

Pulsa Enter. Verás mensajes parecidos a:

```
Installing: Virtual Machine Platform
Virtual Machine Platform has been installed.
Installing: Windows Subsystem for Linux
...
Installing: Ubuntu
Ubuntu has been installed.
The requested operation is successful.
Changes will not be effective until the system is rebooted.
```

El proceso tarda entre 2 y 5 minutos según tu red. Espera a que termine.

Paso 1.4 — Reiniciar el ordenador

Cuando termine, **reinicia el ordenador completamente** (no es un cierre de sesión, es un reinicio normal). Sin reiniciar no funcionará.

Paso 1.5 — Primer arranque de Ubuntu

Tras reiniciar, Windows abre **automáticamente** una ventana negra que dice:

```
Installing, this may take a few minutes...
```

Espera. Cuando termina, te pregunta:

```
Enter new UNIX username:
```

Escribe un nombre de usuario en minúsculas, sin acentos, sin espacios. Por ejemplo: `jose` o `alumno`. Pulsa Enter.

A continuación pide:

```
New password:
```

Escribe una contraseña. **No verás los caracteres que tecleas**: ni asteriscos, ni nada. Es así en Linux por seguridad. Teclea a ciegas y pulsa Enter. Te la pedirá una segunda vez para confirmar.

Cuando termina ves el prompt verde:

```
jose@MIPC:~$
```

Ya estás dentro de Ubuntu. Eso significa: estás en un Linux dentro de Windows.

Si te aparece el error 0x80370102

Te lo dice así:

```
WslRegisterDistribution failed with error: 0x80370102
Please enable the Virtual Machine Platform Windows feature
and ensure virtualization is enabled in the BIOS.
```

Quiere decir que **falta activar la virtualización**. Pasos:

1. **Comprueba la virtualización en el sistema.** Pulsa Ctrl + Shift + Esc para abrir el Administrador de tareas. Ve a la pestaña **Rendimiento** → **CPU**. Mira abajo a la derecha la línea **Virtualización**:
 - Si pone **Habilitado**: el problema es solo de Windows. Sigue al siguiente punto.
 - Si pone **Deshabilitado**: hay que activarla en la BIOS. Salta al punto “Activar virtualización en la BIOS” más abajo.
2. **Activa las características de Windows.** Abre PowerShell como administrador y ejecuta los dos comandos:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-
Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /
all /norestart
```

Cuando acaban (30-60 segundos), **reinicia el ordenador completo**.

3. **Vuelve a lanzar WSL:** tras reiniciar, abre PowerShell normal (no hace falta admin) y ejecuta:

```
wsl --install -d Ubuntu
```

Activar virtualización en la BIOS

Si el Administrador de tareas decía “Deshabilitado”:

1. **Reinicia el ordenador** y aporrea la tecla de BIOS en cuanto veas el logo del fabricante. Las teclas habituales:

Marca	Tecla
HP	F10 o Esc
Dell	F2 o F12
Lenovo	F1, F2 o Enter luego F1
ASUS	F2 o Del
Acer	F2 o Del
MSI	Del
Sobremesa genérico	Del o F2

2. **Busca la opción de virtualización.** Está en una de estas secciones según tu BIOS:

- “Advanced” → “CPU Configuration”
- “Configuration” → “Virtualization”
- “Security” → “Virtualization”
- “System Configuration”

Y el nombre exacto cambia:

- Procesador Intel: **Intel Virtualization Technology** o **VT-x**.
- Procesador AMD: **SVM Mode**, **AMD-V** o **SVM**.

Cámbialo a **Enabled**.

3. **Guarda y sal** (suele ser F10, luego confirmar). El equipo reinicia. Vuelve al Paso 1.5.

A.2 Instalar Docker Desktop

Docker Desktop es la aplicación que gestiona los contenedores.

Paso 2.1 — Descargar Docker Desktop

Abre tu navegador y entra en:

<https://www.docker.com/products/docker-desktop/>

Pulsa el botón **Download for Windows - AMD64** (o **ARM64** si tu PC es ARM, raro). Se descarga un fichero `Docker Desktop Installer.exe` de unos 600 MB.

Paso 2.2 — Ejecutar el instalador

Doble click en el `.exe` descargado. Aparece una ventana de configuración:

- **Use WSL 2 instead of Hyper-V:** déjalo **marcado**. Es lo que quieres.

- **Add shortcut to desktop:** como prefieras.

Pulsa **OK**. La instalación tarda entre 3 y 5 minutos.

Paso 2.3 — Cerrar sesión de Windows

Cuando termina, te pide cerrar sesión. No es un reinicio: solo cerrar sesión y volver a iniciarla con tu usuario. Hazlo.

Paso 2.4 — Abrir Docker Desktop por primera vez

Tras volver a iniciar sesión, abre Docker Desktop desde el menú Inicio. La primera vez:

- Te muestra un acuerdo de licencia: acéptalo (es gratuito para uso personal y educativo).
- Empieza a arrancar el motor. Verás el icono de la **ballena de Docker** en la bandeja del sistema (abajo a la derecha, junto al reloj). Mientras está animada (las olas suben y bajan), Docker está arrancando.

Espera hasta que la ballena se **quede fija** (sin animación). En ese momento, Docker está listo. Suele tardar 1-2 minutos.

Paso 2.5 — Conectar Docker con WSL (importante)

Si has elegido la ruta WSL, Docker Desktop tiene que “ver” tu Ubuntu. Esto se olvida siempre y luego falla todo.

1. Abre Docker Desktop.
2. Engranaje arriba a la derecha (**Settings**).
3. En el menú lateral: **Resources** → **WSL integration**.
4. Marca la casilla “**Enable integration with my default WSL distro**”.
5. En la lista de abajo, encuentra **Ubuntu** y activa su interruptor.
6. Pulsa **Apply & Restart**.

Espera 30 segundos a que Docker reinicie.

Paso 2.6 — Verificar Docker desde Ubuntu

Abre **Ubuntu** desde el menú Inicio (no PowerShell). En el prompt verde, teclea:

```
docker --version
```

Debes ver algo así:

```
Docker version 25.0.3, build 4debf41
```

Después:

```
docker run --rm hello-world
```

Docker descarga una imagen pequeña de prueba y la ejecuta. Al final verás:

```
Hello from Docker!  
This message shows that your installation appears to be working correctly.
```

Si llegas aquí, Docker funciona perfectamente.

A.3 Si no quieres usar WSL: PowerShell nativo

Si has decidido **no** usar WSL, instalas Docker Desktop igual (Paso A.2) pero **NO** marcas la **integración WSL**. Trabajarás siempre desde PowerShell.

Para arrancar PowerShell en tu carpeta del curso, escribe en el menú Inicio “PowerShell” y abre la app **Windows PowerShell** (no la versión “ISE”). Te aparece la ventana azul con prompt:

```
PS C:\Users\TuNombre>
```

Para verificar:

```
docker --version  
docker run --rm hello-world
```

Mismas salidas que antes.

B · macOS (Intel y Apple Silicon)

En macOS solo necesitas Docker Desktop. La terminal viene de serie.

B.1 Identifica tu chip

Manzana arriba izquierda → **Acerca de este Mac**. Mira la línea “Chip” o “Procesador”:

- Si dice **Apple M1, M2, M3 o M4**: tu chip es ARM64 (Apple Silicon).
- Si dice **Intel Core i5/i7/i9** o similar: tu chip es Intel x86_64.

Esto importa para descargar la versión correcta de Docker.

B.2 Instalar Docker Desktop

Opción rápida con Homebrew (recomendado si ya lo tienes)

Si ya usas Homebrew (gestor de paquetes de macOS):

```
brew install --cask docker
```

Termina en 2-3 minutos. Si no tienes Homebrew, ve a la opción siguiente.

Opción manual

Abre tu navegador y entra en:

```
https://www.docker.com/products/docker-desktop/
```

Pulsa:

- **Download for Mac - Apple Silicon** si tu Mac es M1/M2/M3/M4.
- **Download for Mac - Intel** si tu Mac es Intel.

Se descarga un fichero `Docker.dmg` (~500-700 MB). Doble click sobre él. Arrastra el icono de Docker a la carpeta Aplicaciones.

B.3 Primer arranque

Abre Aplicaciones → Docker. La primera vez:

- macOS pide confirmar que quieres abrir una aplicación descargada de internet. Confirma.

- Docker pide permiso para instalar componentes con privilegios de administrador. Introduce tu contraseña de usuario macOS.
- Aparece un acuerdo de licencia. Acéptalo.
- El icono de la **ballena** aparece arriba en la barra de menú (junto al reloj y el wifi). Cuando deja de animarse, Docker está listo.

B.4 Verificar

Abre Terminal (Cmd + Espacio, escribe “Terminal”, Enter):

```
docker --version
docker compose version
docker run --rm hello-world
```

Si los tres comandos responden con texto coherente, Docker funciona.

C • Linux

En Linux instalas el motor de Docker directamente, sin Desktop. Los comandos cambian según tu distribución.

C.1 Ubuntu / Debian

Abre una terminal y ejecuta línea por línea (te pedirá tu contraseña cuando uses `sudo`):

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

El primer comando actualiza la lista de paquetes disponibles. El segundo instala Docker y el plugin de Compose. El tercero arranca el servicio de Docker y lo deja activado para arrancar automáticamente en cada inicio del sistema.

A continuación, añade tu usuario al grupo `docker` para no tener que escribir `sudo` cada vez:

```
sudo usermod -aG docker $USER
```

Para que el cambio de grupo tenga efecto, **cierra la terminal y abre otra nueva**. O alternatively:

```
newgrp docker
```

C.2 Fedora

```
sudo dnf install -y docker docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Cierra y abre la terminal.

C.3 Arch Linux

```
sudo pacman -S --noconfirm docker docker-compose
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Cierra y abre la terminal.

C.4 Verificar

```
docker --version  
docker compose version  
docker run --rm hello-world
```

Los tres deben responder. Si el último imprime “Hello from Docker!”, estás listo.

D • Comprobación final (todos los sistemas)

Independientemente del sistema operativo, abre una terminal donde puedas ejecutar `docker` (en Windows con WSL: Ubuntu; en Windows nativo: PowerShell; en macOS y Linux: la terminal del sistema). Lanza estos cuatro comandos uno tras otro:

```
docker --version
docker compose version
docker info
docker run --rm hello-world
```

Lo que deberías ver:

- `docker --version` : una línea con la versión, por ejemplo `Docker version 25.0.3, build 4deb41`.
- `docker compose version` : una línea con la versión de Compose, por ejemplo `Docker Compose version v2.24.5`.
- `docker info` : un bloque grande con información del motor. Lo importante es que **no haya errores en rojo**. Las primeras líneas deben hablar de containers, imágenes, almacenamiento.
- `docker run --rm hello-world` : descarga una imagen pequeña y la ejecuta. Termina con “Hello from Docker!”.

Si los cuatro funcionan, tu sistema está perfecto para el curso.

E · Descargar y descomprimir el material del curso

Pide a tu profesor el fichero `Anbam-curso.zip`. Lo recibirás por correo, USB o enlace de descarga.

Dónde descomprimirlo

La ruta correcta depende del sistema operativo. Es **importante** elegirla bien porque Docker en algunos casos es muy lento si pones el material en la carpeta equivocada.

Sistema	Ruta recomendada	Por qué
macOS	<code>~/Anbam-curso</code>	Cualquier ubicación del home funciona bien.
Linux	<code>~/Anbam-curso</code>	Igual que macOS.
Windows + WSL	<code>~/Anbam-curso</code> dentro de Ubuntu	NO en <code>/mnt/c/</code> . Docker es muy lento si los volúmenes están en el disco de Windows.
Windows nativo	<code>C:\Anbam-curso</code>	Carpeta corta, sin acentos ni espacios.

Cómo descomprimir, paso a paso

En macOS

1. Localiza el `Anbam-curso.zip` en Finder (suele estar en Descargas).
2. Doble click sobre él. macOS lo descomprime automáticamente y crea una carpeta `Anbam-curso` al lado.
3. Arrástrala a tu home (`/Users/TuNombre/`).
4. Abre Terminal y verifica:

```
cd ~/Anbam-curso
ls
```

Debes ver una lista con `EMPIEZA_AQUI.md`, `setup.sh`, `labs/`, `material_alumno/`, etc.

En Linux

```
cd ~  
unzip ~/Downloads/Anbam-curso.zip  
cd Anbam-curso  
ls
```

Misma salida que en macOS.

En Windows con WSL (recomendado)

El ZIP probablemente lo tienes en Descargas de Windows. Para llevarlo al sistema de archivos de Linux y descomprimirlo allí:

```
cd ~  
cp /mnt/c/Users/$USER/Downloads/Anbam-curso.zip .  
unzip Anbam-curso.zip  
cd Anbam-curso  
ls
```

Si `unzip` no está instalado en Ubuntu:

```
sudo apt install -y unzip
```

Y vuelve a ejecutar el `unzip` anterior.

En Windows nativo (sin WSL)

1. Localiza `Anbam-curso.zip` en el Explorador de archivos. Suele estar en `C:\Users\TuNombre\Downloads\`.
2. Click derecho sobre el ZIP → **Extraer todo**.
3. En la ventana, en “Los archivos se extraerán a esta carpeta”, borra la ruta sugerida y escribe `C:\Anbam-curso`.
4. Pulsa **Extraer**. Windows crea `C:\Anbam-curso\Anbam-curso\...` por defecto (carpeta dentro de carpeta). Mueve el contenido de esa subcarpeta a `C:\Anbam-curso\` para evitar la duplicación.
5. Abre PowerShell:

```
cd C:\Anbam-curso  
ls
```

Debes ver `EMPIEZA_AQUI.md`, `setup.ps1`, `labs/`, etc.

F • Levantar el laboratorio del curso

Desde la carpeta donde descomprimiste el material, ejecuta un único comando:

macOS, Linux y Windows con WSL

```
./setup.sh
```

Windows nativo (PowerShell)

```
.\setup.ps1
```

Si Windows te dice que la ejecución de scripts está deshabilitada, abre PowerShell **como administrador** y ejecuta:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Confirma con **S** (sí). Cierra y vuelve a abrir PowerShell normal, y relanza `.\setup.ps1`.

Qué hace `./setup.sh` (o `setup.ps1`)

El script hace cinco cosas, en este orden:

1. **Comprueba que Docker está abierto y funciona.** Si no, te avisa y para. Si te paras aquí: asegúrate de que la ballena de Docker está fija en la bandeja.
2. **Descarga las cinco imágenes del laboratorio**, con reintentos automáticos si una falla:
 - `postgres:16-alpine` — la base de datos para MLflow.
 - `minio/minio` — almacenamiento tipo S3 para el curso.
 - `minio/mc` — cliente de MinIO, crea los buckets iniciales.
 - `ghcr.io/mlflow/mlflow` — el tracking server.
 - `jupyter/scipy-notebook` — entorno Jupyter con librerías científicas.La primera vez tarda **5-10 minutos** y descarga unos 2 GB. La segunda vez, segundos.
3. **Arranca los contenedores con Docker Compose.** Define la red interna, los volúmenes persistentes y las dependencias entre servicios.
4. **Espera a que cada servicio responda correctamente.** Hace comprobaciones HTTP a cada URL y solo continúa cuando todos están sanos.
5. **Te muestra los enlaces** para abrir en el navegador y te indica el siguiente paso (lanzar el Lab 1).

Lo que debes ver al final

```
=====
LIST0. Abre estos enlaces en tu navegador:
=====

Jupyter   : http://localhost:8888   (token: anban)
MLflow    : http://localhost:5050
MinIO UI  : http://localhost:9001   (minio / minio12345)

Para hacer el Lab 1, ejecuta:
    cd labs/lab1_dataops && ./run.sh
=====
```

Comprobar que los tres servicios están vivos

Abre el navegador y entra a cada URL:

URL	Qué debes ver
http://localhost:8888	Una página de Jupyter pidiendo un token. Mete <code>anban</code> y pulsa Sign in. Aparece el explorador de ficheros de Jupyter.
http://localhost:5050	La interfaz de MLflow. Sin experimentos aún (los crearemos en el Lab 2).
http://localhost:9001	Una ventana de login de MinIO. Usuario <code>minio</code> , contraseña <code>minio12345</code> . Tras entrar, ves los buckets <code>datasets</code> y <code>mlflow</code> .

Si los tres responden, estás dentro del curso. **Felicidades:** la parte difícil ha terminado.

G · Si algo falla: el doctor automático

El curso incluye un script de diagnóstico que detecta y explica los problemas más comunes. Si en cualquier momento algo no funciona:

macOS, Linux y Windows con WSL

```
./doctor.sh
```

Windows nativo

```
.\doctor.ps1
```

Salida típica con todo bien:

```

=====
Diagnostic del laboratorio ANBAN
=====

- Sistema -
  · Sistema operativo: Linux 6.5.0
- Docker instalado -
  ✓ docker encontrado: Docker version 25.0.3
- Motor de Docker activo -
  ✓ el motor responde
- Docker Compose v2 -
  ✓ docker compose disponible
- Acceso a registries (Docker Hub, GHCR) -
  ✓ Docker Hub accesible
  ✓ GHCR (GitHub Container Registry) accesible
- Puertos del laboratorio -
  ✓ puerto 5050 ocupado por un contenedor del curso
  ✓ puerto 5432 ocupado por un contenedor del curso
  ✓ puerto 8888 ocupado por un contenedor del curso
  ✓ puerto 9000 ocupado por un contenedor del curso
  ✓ puerto 9001 ocupado por un contenedor del curso
- Contenedores del curso -
  ✓ anban-postgres arriba
  ✓ anban-minio arriba
  ✓ anban-mlflow arriba
  ✓ anban-jupyter arriba
  ✓ anban-minio-init termino correctamente
- Servicios HTTP del laboratorio -
  ✓ Jupyter responde
  ✓ MLflow responde
  ✓ MinIO API responde
  ✓ MinIO UI responde
- Espacio en disco -
  ✓ 50G disponibles

=====
  Todo en orden. No hay problemas detectados.
=====

```

Si hay algún problema, el doctor te lo dice claramente con una marca **X** y una línea “arreglo:” que te explica qué hacer.

H · Problemas frecuentes y cómo solucionarlos

Docker no arranca o la ballena queda animada eternamente

Causa probable 1: virtualización deshabilitada en BIOS

Es la causa más habitual en Windows. Ve a la sección “Activar virtualización en la BIOS” del capítulo A.

Causa probable 2: Hyper-V deshabilitado por antivirus

Algunos antivirus (Avast, McAfee) o programas de virtualización viejos (VirtualBox antiguo) bloquean Hyper-V. Pasos:

1. Desinstala VirtualBox/VMware si los tienes y no los necesitas.
2. Desactiva temporalmente el antivirus.
3. Reinicia.
4. Vuelve a arrancar Docker Desktop.

Causa probable 3: WSL desactualizado

Solo en Windows. Abre PowerShell como administrador:

```
wsl --update  
wsl --shutdown
```

Y reabre Docker Desktop.

“Cannot connect to the Docker daemon”

Significa que el motor de Docker no está arrancado.

- **macOS / Windows:** abre Docker Desktop manualmente desde el menú Inicio o Aplicaciones.
- **Linux:** arranca el servicio:

```
sudo systemctl start docker
```

Una imagen no se descarga (timeout, 500 Internal Server Error)

Tu red está bloqueando algún registry de Docker.

1. Cierra Docker Desktop (Quit Docker, esperar a que la ballena desaparezca).
2. Reabre Docker Desktop. Espera a que la ballena se fije.

3. Lanza `./setup.sh` (o `.\setup.ps1`) otra vez.

Si sigue fallando: tu red corporativa o de hotel bloquea `*.docker.io` o `*.ghcr.io`. Cambia de red (red de datos del móvil compartida, por ejemplo). Si funciona ahí, ya sabes que era la red. Pide a IT que abra los registries en la red de la oficina.

Puerto ya ocupado por otro proceso

Si al ejecutar `./setup.sh` te aparece un error tipo:

```
Bind for 0.0.0.0:5050 failed: port is already allocated
```

Significa que algún otro programa está usando ese puerto.

Para averiguar cuál:

- **Linux / macOS:**

```
lsof -i :5050
```

- **Windows PowerShell:**

```
Get-NetTCPConnection -LocalPort 5050
```

Cierra la aplicación que esté usando ese puerto y vuelve a lanzar `./setup.sh`. Si necesitas mantener ambos programas, cambia el puerto en `docker/docker-compose.yml` (líneas `ports:`).

El laboratorio funciona pero un alumno tiene una pantalla negra al abrir Jupyter

Tu navegador necesita el token. Cuando abres `http://localhost:8888` te pide un token: introduce `anban` (en minúsculas) y pulsa “Log in”.

Ejercicio para fijar lo aprendido

Para asegurarte de que todo está correcto, vamos a hacer un pequeño ejercicio. En tu terminal, ejecuta:

```
docker ps
```

Tienes que ver **cuatro contenedores** del curso en estado “Up”. Si ves:

NAME	STATUS	PORTS
anban-jupyter	Up 2 minutes	0.0.0.0:8888->8888/tcp
anban-mlflow	Up 2 minutes	0.0.0.0:5050->5000/tcp
anban-minio	Up 2 minutes	0.0.0.0:9000-9001->9000-9001/tcp
anban-postgres	Up 2 minutes	0.0.0.0:5432->5432/tcp

todo está correcto. Anota la columna `STATUS` de cada uno. Ese es el estado normal del laboratorio. Cuando los nombres aparezcan en `Exited`, sabrás que algo se cayó y tendrás que volver a hacer `./setup.sh`.

1. La pregunta con la que empieza todo

Llevas seis meses entrenando un modelo. Lo entregas a producción en diciembre. Pasan tres meses. Alguien te pregunta:

- ¿Con qué versión exacta del dataset se entrenó este modelo?
- ¿Qué hiperparámetros tenía?
- ¿Cuántas predicciones ha hecho desde entonces?
- Si lo necesitamos reentrenar hoy con los mismos datos, ¿podemos?

Si no tienes respuestas claras a estas cuatro preguntas, tu sistema ML sufre **deuda técnica oculta**. Es el problema que describieron Sculley y otros en el famoso paper *Hidden Technical Debt in Machine Learning Systems* (Google, 2015). El paper muestra con un esquema muy gráfico que el código del modelo es una fracción minúscula del sistema real, rodeado de servicios de validación de datos, configuración, monitorización, serving, ingestión, etc.

El código del modelo es el 5 % del sistema. El otro 95 % es plomería. MLOps es la disciplina que se ocupa de esa plomería.

2. Cuatro disciplinas que se confunden

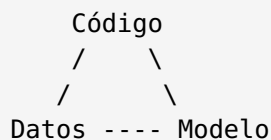
Mucha gente usa estos términos como sinónimos. No lo son. Tener clara la diferencia te ahorra debates innecesarios.

Disciplina	Qué versiona	Qué automatiza
DevOps	Código	Build, test, deploy
DataOps	Datos y transformaciones	Pipelines de ingesta, validación, lineage
MLOps	Código, datos y modelo	Reentrenamiento, despliegue, monitorización del modelo
AIOps	Logs y métricas de infraestructura	Detección de anomalías en sistemas
LLMOps	Prompts, modelos, embeddings y RAG	Evaluación, guardrails, versionado de prompts

DataOps y MLOps son **complementarios**: DataOps termina cuando hay un dataset limpio y reproducible. MLOps empieza ahí. Sin DataOps, MLOps es un castillo de naipes.

3. El triángulo Código-Datos-Modelo

Los tres vértices de cualquier sistema ML están acoplados:



Si cambias uno y no tocas los otros, el sistema deja de ser coherente:

- Cambias el código sin reentrenar: el modelo está desactualizado respecto a lo que el código espera.
- Cambias el dato sin re-evaluar: puede haber degradación silenciosa (drift, ver Módulo 5).
- Cambias el modelo sin versionar el dato: no puedes reproducirlo en el futuro.

Por eso necesitamos versionar los tres. Git resuelve el código. DVC, LakeFS o Delta Lake resuelven los datos. MLflow, Weights & Biases o similares resuelven el modelo y los experimentos.

4. Ciclo de vida: CRISP-DM y CRISP-ML(Q)

CRISP-DM es el ciclo clásico de minería de datos (propuesto en 1996, sigue vigente):

1. Entendimiento del negocio.
2. Entendimiento de los datos.
3. Preparación de los datos.
4. Modelado.
5. Evaluación.
6. Despliegue.

CRISP-ML(Q) es una revisión moderna (Studer et al., 2021) que añade explícitamente:

- **Calidad de datos** como fase propia y continua.
- **Monitorización** posterior al despliegue.
- **Mantenimiento**: drift, retraining, retirada del modelo.

En la práctica, MLOps implementa CRISP-ML(Q) con herramientas concretas. En este curso vas a ver cada herramienta para cada fase.

5. Niveles de madurez MLOps (modelo de Google)

Google publicó un whitepaper muy conocido (*MLOps: Continuous delivery and automation pipelines in machine learning*) con tres niveles de madurez. Es la forma más extendida de diagnosticar dónde está una organización.

5.1 Nivel 0 — Proceso manual

- El data scientist trabaja en un notebook.
- Entrega un fichero `.pkl` o despliega manualmente al servidor.
- No hay tests automáticos del modelo.
- No hay tracking de experimentos.
- Si algo falla, se rehace a mano.

La mayoría de organizaciones está aquí, aunque alguna no lo admita.

5.2 Nivel 1 — Pipeline ML automatizado

- El entrenamiento es un pipeline reproducible, no un notebook.
- Hay validación de datos automática.
- Hay un registry de modelos.
- El despliegue está automatizado.
- Existe monitorización básica.

Llegar aquí ya supone un cambio cultural y técnico importante. Al final de este curso podrás reconocer todas las piezas de un Nivel 1 y reproducirlas.

5.3 Nivel 2 — CI/CD completo

- Cada commit puede disparar entrenamiento, evaluación, promoción y despliegue.
- Reentrenamiento automático ante drift.
- Despliegues canary o blue-green.
- Observabilidad completa con alerting.
- Rollback automático.

Pocas organizaciones tienen este nivel completo, y casi siempre solo para sus modelos críticos.

6. Roles en un equipo MLOps

Rol	Responsabilidad principal
Data Engineer	Pipelines de datos, calidad, almacenamiento
Data Scientist	Modelado, experimentación, feature engineering
ML Engineer	Productivización del modelo, código de calidad
MLOps Engineer	Plataforma, automatización, observabilidad
SRE / ML Platform	Infraestructura, escalado, fiabilidad

En equipos pequeños, una sola persona cubre varios roles. En equipos grandes hay especialización clara. Lo importante es que **alguien** se haga cargo de cada función.

7. Ejercicio: diagnóstico de madurez

Objetivo del ejercicio: poner en práctica el modelo de madurez de Google sobre tres escenarios reales. No necesitas ordenador, lo haces a mano o en grupo.

Lee cada escenario y decide en qué nivel está. Argumenta tu respuesta. Después contrasta con la solución sugerida al final.

Escenario A

Una startup de logística entrena cada semana un modelo de previsión de demanda. El data scientist tiene un notebook, lo ejecuta los lunes, exporta a `.pkl`, lo sube por SFTP al servidor de la API. La API recarga el modelo al reiniciarse. Hay un README de cinco líneas. No hay tests.

Tu diagnóstico: _____

Escenario B

Un banco tiene un modelo de scoring de crédito. Cada commit a `main` dispara un workflow de GitHub Actions que entrena, evalúa contra un dataset golden, registra en MLflow y, si supera al actual, lo promociona a Staging para QA manual. Producción se aprueba por pull request.

Tu diagnóstico: _____

Escenario C

Un retailer entrena un modelo de recomendación cada noche con Airflow, tiene un model registry, monitoring con Evidently, retraining automático si el drift supera un umbral, despliegue canary del 5 %, y un dashboard que su SRE mira a las 9 de la mañana. Si algo falla, rollback automático.

Tu diagnóstico: _____

Solución sugerida

Escenario A: Nivel 0. Es el clásico ML en notebook con despliegue manual. Sin tests, sin tracking, sin reproducibilidad. La mayoría de proyectos al empezar están aquí.

Escenario B: Nivel 1 alto, casi Nivel 2. Tiene CI/CD, registry, evaluación automática y promoción condicional. Le falta CT (continuous training) automático y monitoring de drift para llegar a Nivel 2 completo.

Escenario C: Nivel 2. Tiene los tres bucles (CI, CD, CT), observabilidad completa, estrategia de despliegue gradual y rollback automático.

¿Y tu organización? Si hay otros alumnos en clase, comentadlo en voz alta. Casi siempre la respuesta sincera es “estamos en Nivel 0”. Eso está bien: este curso te enseña a moverte hacia Nivel 1.

8. Vocabulario mínimo

Término	Significado
Reproducibilidad	Capacidad de obtener exactamente el mismo modelo dadas las mismas entradas
Experiment	Conjunto de ejecuciones de entrenamiento para un mismo problema
Run	Una ejecución concreta dentro de un experiment
Model Registry	Almacén versionado de modelos con stages (Staging, Production, Archived)
Pipeline	Grafo de pasos automatizado (DVC, Airflow, Prefect, Dagster)
Drift	Cambio en la distribución de inputs (data drift) o de la relación input-output (concept drift)
Feature Store	Almacén compartido de features para entrenamiento y servicio
Model Card	Documento estandarizado que describe propósito, datos y limitaciones de un modelo
Audit trail	Registro completo y trazable de qué predicciones hizo cada modelo, cuándo y con qué inputs

9. Para profundizar

Si quieres ir más allá:

- Sculley et al., *Hidden Technical Debt in Machine Learning Systems* (NeurIPS 2015). Es el paper fundacional.
- Studer et al., *Towards CRISP-ML(Q)* (2021). Para entender el ciclo de vida moderno.
- Google Cloud, *MLOps: Continuous delivery and automation pipelines in machine learning* (whitepaper). Es donde se define el modelo de madurez.
- **ml-ops.org**. Hoja de ruta de la comunidad MLOps.
- **Made With ML** (<https://madewithml.com>). Tutoriales gratuitos excelentes.

1. Qué es DataOps y por qué te importa

Antes de entrar en herramientas, asegurémonos de entender qué es DataOps. La definición corta:

DataOps es la disciplina que aplica las prácticas de DevOps al ciclo de vida del dato.

Eso significa: automatización, versionado, observabilidad y calidad desde el momento en que el dato entra al sistema hasta que llega al modelo. En la práctica, cuatro principios:

- 1. Orquestación.** Defines tu pipeline de datos como un grafo declarativo (ingesta → limpieza → transformación → validación → publicación). Un solo comando lo reproduce entero.
- 2. Observabilidad.** En cada momento sabes en qué paso del pipeline estás, cuánto ha tardado, qué datos ha producido y si algún paso ha fallado.
- 3. Automatización.** Si tienes que ejecutar un paso a mano cada vez, antes o después se te olvidará. Todo se automatiza.
- 4. Versionado.** Datos, código y parámetros. Los tres tienen que poder volverse atrás juntos a un estado anterior cualquiera.

En este módulo vamos a centrarnos en los puntos 1 (orquestación) y 4 (versionado) usando **DVC**, y en el punto de calidad de datos con una implementación inspirada en **Great Expectations**.

2. La herramienta principal: DVC

2.1 Qué es DVC

DVC son las siglas de Data Version Control. Es una herramienta de código abierto que extiende Git para que pueda manejar ficheros muy grandes sin almacenarlos directamente en el repositorio. Su modelo mental es sencillo:

- En **Git** se guarda solo un puntero pequeño (un fichero `.dvc` de unos pocos bytes) que contiene el hash criptográfico del dato real.
- El **dato real** (que puede pesar gigabytes) se guarda en un almacenamiento externo: un bucket S3, un MinIO local, Google Cloud Storage, Azure Blob, un disco compartido, etc.

Cuando un compañero clona el repositorio, Git le baja solo los punteros. Para recuperar los datos reales ejecuta `dvc pull` y DVC los descarga del almacenamiento externo usando los hashes como referencia.

2.2 Por qué no basta con Git para datos

Git se diseñó para texto plano. Tres problemas si guardas datos en Git:

1. **Crece sin control.** Git almacena cada versión del binario entera, no diferencias. Un CSV de 500 MB que cambias diez veces ocupa 5 GB en el repositorio.
2. **Los clones se vuelven imposibles.** Cualquier `git clone` baja todo el histórico.
3. **GitHub bloquea ficheros grandes.** El límite duro está en 100 MB por fichero, y a partir de 50 MB ya recibes advertencias.

DVC resuelve estos tres problemas a la vez: Git solo trackea metadatos, los datos viven aparte.

2.3 Alternativas a DVC

No es la única herramienta. Te interesa conocer las opciones:

Herramienta	Idea	Cuándo elegirla
DVC	Puntero en git + blob en remoto	Equipos pequeños/medianos, integración con Git, on-premises
LakeFS	Git para data lakes enteros	Si ya trabajas con S3/GCS a escala
Delta Lake	Tablas versionadas sobre Spark/Databricks	Stack Databricks
Apache Iceberg	Formato de tabla con snapshots	Data lakes modernos, OLAP

En este curso usamos DVC porque encaja con Git, funciona en local con MinIO sin necesidad de cloud, y enseña los conceptos generales muy bien.

2.4 Instalación de DVC

DVC se instala con `pip` (el gestor de paquetes de Python). Como en el curso usamos Docker, **no tienes que instalarlo en tu sistema**: el script `run.sh` lo instala dentro de un entorno aislado cuando ejecutas el lab. Pero si quieres tenerlo a mano fuera del lab:

macOS, Linux, Windows con WSL

Abre tu terminal y ejecuta:

```
pip install "dvc[s3]==3.55.2"
```

La sintaxis `dvc[s3]` significa “instala DVC con el extra `s3`”. Ese extra es el plugin que permite hablar con MinIO y otros almacenes compatibles con S3. Sin él, solo podrías usar DVC con discos locales.

Verifica:

```
dvc --version
```

Debe imprimir algo como `3.55.2`.

Windows nativo

Si no usas WSL, primero asegúrate de tener Python instalado. Abre PowerShell:

```
python --version
```

Si no responde, instala Python desde Microsoft Store buscando “Python 3.11” o desde python.org.
Después:

```
pip install "dvc[s3]==3.55.2"  
dvc --version
```

3. La herramienta secundaria: Great Expectations (y su alternativa Pandera)

3.1 Qué es Great Expectations

Great Expectations (GE) es una librería de Python para escribir “expectations”: reglas que tus datos deben cumplir. Por ejemplo:

- La columna `edad` debe tener valores entre 0 y 120.
- La columna `email` no puede tener valores nulos.
- La columna `pais` solo puede tomar valores de un catálogo cerrado.

Cuando un dataset llega a tu pipeline, lo pasas por GE y si alguna expectation falla, el pipeline se detiene. Esto evita que datos malos se propaguen aguas abajo y rompan modelos en producción.

3.2 Por qué no usamos Great Expectations directamente

GE es muy potente pero su API cambia mucho entre versiones. En un curso de dos días, perderíamos demasiado tiempo peleándonos con la configuración. Por eso vamos a implementar las mismas ideas con **pandas puro**: el código es más simple, sin librerías que cambien, y los conceptos son idénticos.

Si quieres usar GE de verdad después del curso, los conceptos se traducen uno a uno.

3.3 La alternativa ligera: Pandera

Otra opción muy popular es **Pandera**, una librería más ligera que GE pero con la misma idea. Se integra muy bien con pandas y polars. Definición de schema con Pandera:

```
import pandera as pa

schema = pa.DataFrameSchema({
    "age": pa.Column(int, pa.Check.in_range(0, 120)),
    "email": pa.Column(str, nullable=False),
})
```

No la usamos en el lab para no añadir dependencias, pero es buena opción para tus proyectos.

4. Las cinco dimensiones de calidad del dato

Antes de empezar a trackear datos, conviene tener un marco mental para pensar en su calidad. Cinco dimensiones que cualquier proyecto serio debería medir:

Dimensión	Pregunta que responde
Compleitud	¿Cuántos valores nulos hay donde no debería haberlos?
Validez	¿Los valores están dentro del rango o dominio esperado?
Unicidad	¿Existen duplicados?
Consistencia	¿Cuadran los datos entre fuentes?
Freshness (frescura)	¿El dato es de cuándo dice ser?

En el lab vamos a comprobar **completitud** (no hay nulos en ninguna columna), **validez** (rangos clínicos plausibles para `age`, `ejection_fraction`, `serum_creatinine`, etc.), **dominio** (las columnas binarias como `anaemia`, `diabetes`, `sex` solo pueden ser 0 o 1) y **schema** (las 13 columnas del dataset están presentes).

5. Antes de la práctica: requisitos

Para hacer el lab necesitas:

1. **El laboratorio del curso arrancado** (`./setup.sh` ya ejecutado). Si no, vuelve al módulo 0.
2. **Una terminal abierta** en la carpeta `Anbam-curso/`.
3. **La carpeta `labs/lab1_dataops/`** con los ficheros del lab.

Comprueba que tienes lo que hace falta:

```
cd labs/lab1_dataops
ls
```

Deberías ver:

```
params.yaml  README.md  run.ps1  run.sh  src
```

Si todo está, sigue al ejercicio.

6. Ejercicio: montar un pipeline reproducible con DVC

Objetivo del ejercicio: practicar el ciclo entero de DVC partiendo de un repositorio vacío. Al terminar tendrás un pipeline declarativo que valida y preprocesa el dataset Heart Failure (UCI 2020), con el dato versionado por DVC y los artefactos generados en parquet.

6.1 Cómo hacer este ejercicio

Tienes dos formas:

- **Guiada con `run.sh`**. El script te lleva paso a paso, ejecutando cada comando y explicándolo. Tú lees y pulsas Enter. Recomendado para la primera vez.
- **A mano**. Sigues los pasos de este capítulo tecleando cada comando tú. Recomendado para la segunda vez, cuando quieras entender qué hace cada línea.

Las dos formas hacen exactamente lo mismo.

6.2 Paso 1 — Inicializar Git

DVC vive dentro de Git: extiende Git, no lo reemplaza. Por eso lo primero es crear un repositorio Git en la carpeta del lab.

```
git init -q
```

Verifica:

```
ls -la .git
```

Tiene que existir una carpeta `.git`. Esa es la carpeta interna de Git con todo el histórico.

A continuación configura tu identidad (Git la pide para los commits):

```
git config --local user.email "alumno@anban.es"  
git config --local user.name "Alumno ANBAN"
```

El flag `--local` significa que esta configuración solo aplica en este repositorio, no en otros que tengas.

Haz un primer commit con lo que ya hay:

```
git add -A  
git commit -m "snapshot inicial"
```

`git add -A` añade todos los ficheros al “staging area”. `git commit` crea un punto en el histórico con esos cambios.

6.3 Paso 2 — Inicializar DVC

```
dvc init -q
```

El flag `-q` significa “modo silencioso”. DVC crea una carpeta `.dvc/` y un fichero `.dvcignore`. Mira lo que aparece:

```
ls -la
```

Verás `.dvc/` y `.dvcignore`. Estos son los ficheros que DVC necesita para funcionar. Los commiteamos al repositorio:

```
git add .dvc .dvcignore
git commit -m "inicializa dvc"
```

6.4 Paso 3 — Configurar el remoto MinIO

DVC necesita saber dónde guardar los blobs grandes. Vamos a decirle que use el bucket `datasets` de nuestro MinIO local.

```
dvc remote add -d minio s3://datasets/lab1
```

Descomponemos este comando:

- `dvc remote add` : registra un nuevo remoto.
- `-d` : marca este remoto como el “default” (el que se usa si no se especifica otro).
- `minio` : es el nombre que le damos al remoto (puedes ponerle el que quieras).
- `s3://datasets/lab1` : la URL del remoto. El protocolo `s3://` funciona con MinIO porque MinIO habla S3 nativo.

A continuación le decimos a DVC dónde está el servidor MinIO:

```
dvc remote modify minio endpointurl http://localhost:9000
```

Esto modifica la propiedad `endpointurl` del remoto `minio`. Lo guarda en `.dvc/config`, que **sí se commitea en Git** (no contiene secretos, solo la URL).

Y ahora las credenciales:

```
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
```

Fíjate en el flag `--local`. Eso es **muy importante**: con `--local` DVC guarda esta configuración en `.dvc/config.local`, que está en `.gitignore` y **no se commitea**. Las credenciales nunca, jamás, bajo ningún concepto deben terminar en Git.

Verifica:

```
cat .dvc/config
cat .dvc/config.local
```

El primero tiene la URL pública. El segundo tiene las credenciales. El primero está en Git, el segundo está ignorado.

Commitea el config público:

```
git add .dvc/config
git commit -m "remoto minio configurado"
```

6.5 Paso 4 — Trackear el dataset

Vamos a copiar el dataset del curso a una carpeta `data/raw/` y decirle a DVC que lo trackee.

```
mkdir -p data/raw
cp ../../datasets/heart_failure/heart_failure.csv data/raw/heart_failure.csv
```

`mkdir -p` crea la carpeta y todas las intermedias si hace falta. El `cp` copia el dataset desde la carpeta común `datasets/` del curso.

Verifica:

```
ls -lh data/raw/
```

Debe aparecer `heart_failure.csv` con un tamaño de unos 3,8 MB.

Ahora viene el comando clave de DVC:

```
dvc add data/raw/heart_failure.csv
```

¿Qué hace este comando? Tres cosas:

1. Calcula el hash MD5 del fichero.
2. Mueve el fichero a la **caché interna de DVC** (en `.dvc/cache/`), nombrándolo por su hash.

3. Crea en su lugar un **puntero**: el fichero `heart_failure.csv.dvc` que contiene los metadatos del original.

Mira los ficheros que ha creado:

```
ls -la data/raw/  
cat data/raw/heart_failure.csv.dvc  
cat data/raw/.gitignore
```

El `.dvc` contiene el hash y el tamaño. El `.gitignore` excluye el CSV real para que no se cuele en Git. Todo esto lo hace DVC automáticamente.

Commitea el puntero:

```
git add data/raw/heart_failure.csv.dvc data/raw/.gitignore  
git commit -m "trackea dataset"
```

Ahora súbelo al remoto:

```
dvc push
```

Esto sube los blobs desde la caché local a MinIO. Verifica abriendo `http://localhost:9001` en el navegador: entra al bucket `datasets/`, luego en `lab1/`, y verás una carpeta con un fichero nombrado por su hash. Eso es DVC en su mínima expresión.

6.6 Paso 5 — Escribir las expectations

El fichero `src/ge_validate.py` ya está en el repositorio. Vamos a mirar su estructura:

```
cat src/ge_validate.py
```

El fichero define cinco funciones de validación:

```

def expect_columns(df):
    # Las 13 columnas esperadas están presentes
    missing = set(EXPECTED_COLUMNS) - set(df.columns)
    assert not missing, f"faltan columnas: {missing}"

def expect_no_nulls(df):
    # Ninguna columna tiene valores nulos
    nulls = df.isna().sum()
    bad = nulls[nulls > 0]
    assert bad.empty, f"hay nulos: {bad.to_dict()}"

def expect_numeric_ranges(df):
    # Cada columna numérica está en su rango clínico plausible
    for col, (lo, hi) in NUMERIC_RANGES.items():
        bad = df[(df[col] < lo) | (df[col] > hi)]
        assert bad.empty, f"{len(bad)} filas con {col} fuera de [{lo}, {hi}]"
    ""

def expect_binary_domain(df):
    # Las columnas binarias solo tienen 0 o 1
    for col in BINARY_COLUMNS:
        extras = set(df[col].unique()) - {0, 1}
        assert not extras, f"{col} con valores fuera de 0/1: {extras}"

def expect_target_balance(df):
    # El target no está demasiado desbalanceado
    positives = df["DEATH_EVENT"].mean()
    assert 0.05 < positives < 0.95, "target desbalanceado"

```

Cada función es una **expectation**: si todo está bien, no hace nada; si algo falla, lanza una excepción que detiene el pipeline.

Ejecuta el validador a mano para ver qué pasa con el dataset bueno:

```
python src/ge_validate.py data/raw/heart_failure.csv
```

Salida esperada:

```

[OK] schema (las 13 columnas esperadas están presentes)
[OK] sin valores nulos en ninguna columna
[OK] rangos clínicos plausibles en 7 columnas
[OK] dominio binario correcto en 6 columnas
[OK] target balanceado razonablemente (positives=32.1%)

TODAS LAS EXPECTATIVAS PASADAS

```

6.7 Paso 6 — Crear el pipeline

DVC tiene su propio formato declarativo para pipelines: el fichero `dvc.yaml`. Vamos a crearlo. Abre tu editor (puede ser `nano`, `vim`, VS Code, lo que prefieras) y crea `dvc.yaml` con este contenido:

```
stages:
  validate:
    cmd: python src/ge_validate.py data/raw/heart_failure.csv
    deps:
      - data/raw/heart_failure.csv
      - src/ge_validate.py
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - data/raw/heart_failure.csv
      - src/preprocess.py
      - params.yaml
    outs:
      - data/processed/train.parquet
      - data/processed/test.parquet
```

Vamos a entender qué dice este YAML:

- **stages:** abre la lista de etapas (stages) del pipeline.
- **validate:** primera etapa, llamada “validate”.
 - **cmd:** comando que se ejecuta cuando se lanza esta etapa.
 - **deps:** dependencias. Si cualquiera de estos ficheros cambia, la etapa se vuelve a ejecutar; si no cambia, se salta.
- **preprocess:** segunda etapa. Depende del CSV crudo, del script de preprocesado y de los parámetros.
 - **outs:** salidas. DVC trackea automáticamente estos ficheros como outputs de esta etapa.

Ejecuta el pipeline entero:

```
dvc repro
```

DVC mira las dependencias, ejecuta primero `validate` y luego `preprocess`, y genera `data/processed/train.parquet` y `data/processed/test.parquet`. Verifica:

```
ls -lh data/processed/
```

6.8 Paso 7 — Ver la caché por dependencias en acción

Vuelve a ejecutar el pipeline:

```
dvc repro
```

Esta vez verás algo así:

```
'data/raw/heart_failure.csv.dvc' didn't change, skipping
Stage 'validate' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Data and pipelines are up to date.
```

DVC ha comparado los hashes de las dependencias con los de la ejecución anterior. Como nada ha cambiado, no ejecuta nada. Esto es la **caché por dependencias**, y en pipelines grandes ahorra horas.

Cambia un parámetro y observa qué pasa. Edita `params.yaml` (con `nano params.yaml` por ejemplo) y modifica `test_size` de `0.2` a `0.3`. Guarda. Ejecuta:

```
dvc repro
```

Ahora dice:

```
Stage 'validate' didn't change, skipping
Running stage 'preprocess'
```

Solo se reejecuta `preprocess`, porque solo cambió una dependencia suya (`params.yaml`). `validate` se salta. En pipelines de decenas de etapas, esta inteligencia ahorra mucho tiempo.

Devuelve `test_size` a `0.2` y ejecuta `dvc repro` para dejar el estado limpio.

6.9 Paso 8 — Romper el dato a propósito

Esta es la parte más importante del ejercicio. Vamos a corromper el CSV y comprobar que el validador nos protege.

Edita la línea 5 del CSV. La forma rápida desde la terminal:

```
python3 -c "
with open('data/raw/heart_failure.csv') as f: lines = f.readlines()
parts = lines[4].split(',')
parts[0] = '200'
lines[4] = ','.join(parts)
with open('data/raw/heart_failure.csv', 'w') as f: f.writelines(lines)
print('Linea 5 modificada: age=200')
"
```

Ahora ejecuta solo el validador. Queremos ver el fallo:

```
python src/ge_validate.py data/raw/heart_failure.csv
```

Debe fallar con:

```
[OK] schema (las 13 columnas esperadas están presentes)
[OK] sin valores nulos en ninguna columna
[FALLO] 1 filas con age fuera de [18, 110]; ejemplos: [200.0]
```

Imagina que el dato malo viniera de un paciente real: si el validador no estuviera ahí, el pipeline lo procesaría, entrenaría un modelo de riesgo cardiovascular con esa basura, y el modelo iría a producción. **En un sistema de salud el coste de eso es enorme.** El validador es el primer cinturón de seguridad.

6.10 Paso 9 — Recuperar el dato bueno

Para recuperar el dato original, lo copiamos otra vez de la fuente y volvemos a trackearlo:

```
cp ../../datasets/heart_failure/heart_failure.csv data/raw/heart_failure.csv

dvc add data/raw/heart_failure.csv
git add data/raw/heart_failure.csv.dvc
git commit -m "restaura dataset original"
dvc repro
```

`dvc repro` debe volver a pasar limpio.

6.11 Paso 10 — Simular un compañero nuevo

Esto demuestra el poder del flujo. Vamos a clonar el repositorio en otra carpeta como si fueras un compañero recién incorporado al proyecto.

```
cd /tmp
git clone ~/Anbam-curso/labs/lab1_dataops nuevo_companero
cd nuevo_companero
ls data/raw/
```

Verás `heart_failure.csv.dvc` y `.gitignore`, pero **no** el CSV grande. Solo el puntero. Ahora recupera los datos:

```
dvc pull
ls -lh data/raw/
```

Ahora sí está el `heart_failure.csv` real, descargado desde MinIO. Reproduce el pipeline:

```
dvc repro
```

Y tienes los parquet generados, exactamente iguales a los originales. Cualquier máquina con acceso al remoto puede reconstruir el dataset y los artefactos exactos a partir del repositorio.

7. Comandos de DVC explicados uno por uno

Como referencia rápida, aquí tienes todos los comandos de DVC que hemos usado, cada uno con su explicación:

Comando	Qué hace
<code>dvc init</code>	Inicializa DVC en un repositorio Git existente. Crea <code>.dvc/</code> y <code>.dvcignore</code> .
<code>dvc add <fichero></code>	Trackea un fichero con DVC. Lo mueve a la caché, crea el puntero <code>.dvc</code> y añade el original al <code>.gitignore</code> .
<code>dvc remote add -d <nombre> <url></code>	Registra un remoto y lo marca como por defecto.
<code>dvc remote modify <nombre> <opción> <valor></code>	Modifica una opción del remoto. Se guarda en <code>.dvc/config</code> (commiteable).
<code>dvc remote modify --local <nombre> <opción> <valor></code>	Como el anterior pero guarda en <code>.dvc/config.local</code> (NO commiteable). Para credenciales.
<code>dvc push</code>	Sube los blobs desde la caché local al remoto.
<code>dvc pull</code>	Descarga los blobs del remoto a la caché local y los enlaza al workspace.
<code>dvc checkout</code>	Sincroniza los ficheros del workspace con los hashes de los <code>.dvc</code> . Útil para revertir cambios.
<code>dvc status</code>	Muestra qué dependencias han cambiado y qué stages necesitan reejecutarse.
<code>dvc repro</code>	Ejecuta el pipeline. Reutiliza las etapas cuyas dependencias no han cambiado.
<code>dvc dag</code>	Imprime el grafo del pipeline (qué stage depende de qué).

8. Pandas y scikit-learn: las librerías de soporte

Los scripts `ge_validate.py` y `preprocess.py` usan dos librerías que conviene conocer:

8.1 Pandas

Pandas es la librería estándar de Python para manipular datos tabulares (filas y columnas). Su tipo principal es el `DataFrame`, muy parecido a una hoja de Excel pero programable.

Las operaciones de pandas que hemos usado en el lab:

```
# Leer un CSV con cabecera (el original ya la incluye):
df = pd.read_csv(path)

# Filtrar filas que se salen de un rango clínico:
bad = df[(df["age"] < 18) | (df["age"] > 110)]

# Contar nulos por columna:
df.isna().sum()

# Estadísticas resumen:
df.describe()

# Balance de la clase objetivo:
df["DEATH_EVENT"].value_counts(normalize=True)

# Eliminar una columna (por ejemplo, una que sea data leakage):
df = df.drop(columns=["time"])
```

Si no tienes pandas instalado en tu sistema (en el lab lo instala el `run.sh`):

```
pip install pandas pyarrow
```

`pyarrow` es necesario para escribir/leer ficheros parquet.

8.2 Scikit-learn

Scikit-learn (importado como `sklearn`) es la librería estándar de machine learning clásico en Python. En este módulo solo usamos una función:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

Esta función divide los datos en dos conjuntos: entrenamiento (80 %) y test (20 %). El parámetro `stratify=y` asegura que la proporción de clases sea igual en ambos conjuntos (importante para problemas desequilibrados).

Para instalarla:

```
pip install scikit-learn
```

9. Checklist de cierre

Antes de pasar al módulo siguiente, comprueba que puedes responder “sí” a todo:

- ¿Entiendes la diferencia entre lo que Git guarda y lo que DVC guarda?
- ¿Sabes para qué sirve `.dvc/config.local` y por qué no se commitea?
- ¿Has visto que `dvc repro` reutiliza etapas con caché?
- ¿Has roto el dato a propósito y has visto el fallo de validación?
- ¿Has hecho `dvc push` y has visto el blob en MinIO?
- ¿Tienes `data/processed/train.parquet` y `test.parquet` generados?

Si la respuesta es sí a todas, pasas al módulo 3 (MLflow).

10. Para profundizar

Si quieres ir más allá de lo que hemos visto:

- **DVC documentation:** <https://dvc.org/doc>. La sección “Get Started” cubre lo mismo que hemos hecho. La sección “User Guide” entra en pipelines complejos, métricas y experimentos.
- **Great Expectations:** <https://greatexpectations.io>. Si tu empresa tiene catálogo de datos serio.
- **Pandera:** <https://pandera.readthedocs.io>. La alternativa ligera, muy recomendable.
- **LakeFS:** <https://lakefs.io>. Cuando subas a escala de TB.

1. El problema que resuelve el tracking de experimentos

Imagina una sesión típica de modelado sin disciplina. Pruebas un hiperparámetro, sale F1 0,72. Cambias otro, sale 0,74. Cambias otro, 0,71. Vuelves a una combinación anterior, sale 0,76. Lo apuntas en un post-it. Dos horas después, ya no recuerdas qué configuración exacta dio el 0,76, ni qué dataset tenías cargado, ni qué seed pusiste.

Multiplica esto por un equipo de cinco personas durante seis meses y tienes el escenario habitual de cualquier proyecto de machine learning sin una herramienta de tracking.

Un sistema de experiment tracking resuelve tres problemas reales:

1. **Reproducibilidad.** Si guardas parámetros, datos, seed y entorno, puedes recrear el modelo exacto en cualquier momento.
2. **Comparación.** Cuando tienes 30 experimentos, ¿cuál era el bueno y por qué? Sin tracking, vuelves a entrenar todos.
3. **Auditoría.** En sectores regulados (banca, salud, seguros) te van a pedir el log completo de cómo entrenaste cada modelo. Sin tracking, te quedas en blanco.

2. La herramienta principal: MLflow

2.1 Qué es MLflow

MLflow es una plataforma de código abierto para gestionar el ciclo de vida del modelo. Está desarrollada por Databricks pero es completamente independiente y autohospedable. Tiene cuatro componentes:

Componente	Para qué sirve
MLflow Tracking	Registra params, métricas, tags y artefactos de cada ejecución de entrenamiento.
MLflow Models	Estándar para empaquetar modelos, con sus metadatos y entorno.
MLflow Projects	Empaquetado reproducible de proyectos ML. Lo veremos solo de pasada.
MLflow Model Registry	Almacén versionado de modelos con stages (Staging, Production, Archived).

En este curso usamos **Tracking, Models y Registry**. Projects es opcional y se usa menos.

2.2 Vocabulario clave

Aprende este vocabulario porque lo usaremos a partir de ahora:

Término	Significado
Experiment	Carpeta lógica que agrupa runs de un mismo problema. Por ejemplo: <code>heart-failure-classifier</code> .
Run	Una ejecución concreta de entrenamiento. Tiene su propio ID.
Parameter	Valor fijado antes del entrenamiento (hiperparámetro, configuración).
Metric	Valor medido. Puede ser un número final o una serie temporal (por epoch).
Tag	Etiqueta libre clave/valor (ej: <code>git_commit=abc123</code>).
Artifact	Fichero asociado al run (modelo serializado, gráficas, datasets).
Signature	Esquema de inputs/outputs del modelo.
Model Registry	Capa por encima donde un modelo tiene nombre, versión y stage.

Los **stages** de un modelo en el Registry son: `None` (recién registrado), `Staging` (en pruebas), `Production` (sirviendo), `Archived` (retirado).

2.3 Alternativas a MLflow

Antes de comprometerte con MLflow, conoce el panorama:

Herramienta	Punto fuerte	Punto débil
MLflow	Open source, autohospedable, estándar	UI mejorable
Weights & Biases	UI excelente, colaboración fluida	SaaS, on-premises de pago
Neptune	Bueno para deep learning	Comunidad más pequeña
Comet	Muy completo	Comercial
Aim	Open source, rápido	Menos features que MLflow

En empresas con restricciones de datos (banca, salud, RGPD) suele ganar MLflow. En equipos de research donde el presupuesto no es problema, suele ganar Weights & Biases.

2.4 Instalación de MLflow

En el curso MLflow ya está corriendo dentro de un contenedor Docker. No tienes que instalarlo. Pero si quieres usarlo localmente fuera del laboratorio:

macOS, Linux, Windows con WSL

```
pip install mlflow==2.17.2
```

Windows nativo

```
pip install mlflow==2.17.2
```

Verifica:

```
mlflow --version
```

Para arrancar un servidor MLflow local manualmente (no necesario en este curso, MLflow ya corre dentro de Docker):

```
mlflow server --backend-store-uri sqlite:///mlflow.db --host 0.0.0.0 --port  
5000
```

3. Las librerías auxiliares de este módulo

3.1 Scikit-learn (modelo “logreg” y “rf”)

Ya la introdujimos en el módulo 2. Aquí la usamos para entrenar dos modelos clásicos:

- **LogisticRegression**: clasificador lineal, rápido, interpretable.
- **RandomForestClassifier**: conjunto de árboles, mejor precisión, más lento.

Para instalarla:

```
pip install scikit-learn==1.5.2
```

3.2 XGBoost (modelo “xgb”)

XGBoost es una librería especializada en gradient boosting: modelos que construyen árboles de decisión de forma iterativa, cada uno corrigiendo los errores del anterior. Suele dar los mejores resultados en problemas tabulares y es el que casi siempre gana en competiciones de Kaggle.

Para instalarla:

```
pip install xgboost==2.1.2
```

En el lab se instala automáticamente. Si tienes problemas en Mac M1:

```
pip install xgboost --no-binary :all:
```

3.3 Boto3 (cliente S3 para MinIO)

Boto3 es el SDK oficial de AWS para Python. Sirve para hablar con S3, y también con MinIO porque MinIO implementa la API de S3. MLflow usa boto3 por debajo cuando sube artefactos al almacén.

Para instalarla:

```
pip install boto3==1.35.49
```

Lo importante para boto3 son las variables de entorno:

Variable	Valor en este curso
AWS_ACCESS_KEY_ID	minio
AWS_SECRET_ACCESS_KEY	minio12345
MLFLOW_S3_ENDPOINT_URL	http://localhost:9000

4. Antes de la práctica: requisitos

Para hacer este lab necesitas:

1. **Haber completado el lab 1.** Tienes que tener generados los ficheros `labs/lab1_dataops/data/processed/train.parquet` y `test.parquet` . Si no los tienes, vuelve al módulo 2.
2. **El laboratorio del curso arrancado.** Verifica que MLflow responde en `http://localhost:5050` y MinIO en `http://localhost:9001`.
3. **Estar en la carpeta** `labs/lab2_mlflow_dvc/` .

5. Ejercicio: trackear 3 modelos y registrar el mejor

Objetivo del ejercicio: entrenar tres modelos diferentes, comparar sus métricas en MLflow, registrar el ganador en el Model Registry y promocionarlo al stage Staging para que el Lab 3 lo pueda cargar.

5.1 Paso 1 — Exportar las variables de entorno

MLflow y boto3 leen su configuración de variables de entorno. Hay que exportarlas en la terminal donde vas a ejecutar los scripts.

macOS, Linux, Windows con WSL

```
export MLFLOW_TRACKING_URI=http://localhost:5050
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

Windows nativo (PowerShell)

```
$env:MLFLOW_TRACKING_URI = "http://localhost:5050"
$env:MLFLOW_S3_ENDPOINT_URL = "http://localhost:9000"
$env:AWS_ACCESS_KEY_ID = "minio"
$env:AWS_SECRET_ACCESS_KEY = "minio12345"
```

Explicación de cada variable:

- **MLFLOW_TRACKING_URI** : dónde está el tracking server. En el curso, en localhost:5050.
- **MLFLOW_S3_ENDPOINT_URL** : dónde está el almacén de artefactos. Como usamos MinIO, le decimos su URL.
- **AWS_ACCESS_KEY_ID** y **AWS_SECRET_ACCESS_KEY** : credenciales para hablar con MinIO via boto3.

Truco práctico: guarda estas líneas en un fichero `setenv.sh` y ejecuta `source setenv.sh` cada vez que abras una terminal nueva.

5.2 Paso 2 — Repasar el script de entrenamiento

Abre `src/train.py`. Las partes clave son:

```

# 1. Conectar con el tracking server
mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])
mlflow.set_experiment("heart-failure-classifier")

# 2. Abrir un run nuevo
with mlflow.start_run(run_name=args.model):
    # 3. Guardar los hiperparámetros
    mlflow.log_params(params)

    # 4. Guardar metadatos como tags
    mlflow.set_tags({
        "git_commit": git_commit(),
        "dataset_version": "lab1-v1",
        "owner": args.owner,
        "framework": type(model).__name__,
    })

    # 5. Entrenar el modelo
    model.fit(X_tr, y_tr)

    # 6. Calcular métricas en test
    metrics = {
        "accuracy": accuracy_score(y_te, y_pred),
        "f1": f1_score(y_te, y_pred),
        "precision": precision_score(y_te, y_pred),
        "recall": recall_score(y_te, y_pred),
        "roc_auc": roc_auc_score(y_te, y_proba),
    }
    mlflow.log_metrics(metrics)

    # 7. Guardar el modelo con su signature
    sig = mlflow.models.infer_signature(X_tr, y_pred)
    mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path="model",
        signature=sig,
        input_example=X_tr.head(3),
        pip_requirements=pip_reqs,
    )

```

Explicación bloque a bloque:

- **set_tracking_uri**: le dice al cliente Python a qué tracking server hablar. Sin esto, MLflow usa una carpeta local.
- **set_experiment**: si el experimento “heart-failure-classifier” no existe, lo crea. Si existe, lo selecciona.
- **start_run**: abre un run nuevo dentro del experimento. El bloque `with` se asegura de cerrarlo aunque el código falle.

- **log_params**: guarda los hiperparámetros del modelo (n_estimators, max_depth, learning_rate, etc.) como diccionario.
- **set_tags**: igual que log_params pero para metadatos arbitrarios. Aquí guardamos el commit de git, la versión del dataset y el usuario que lanzó el run. Cuando audites el modelo en el futuro, estos tags son oro.
- **fit**: entrenamiento estándar de sklearn.
- **log_metrics**: guarda las métricas calculadas en test.
- **log_model**: serializa el modelo, lo sube a MinIO y deja en el run un puntero. La **signature** es el esquema de inputs/outputs: sin esto, cuando luego sirvas el modelo en una API, no sabrías qué columnas debe recibir.

5.3 Paso 3 — Entrenar los tres modelos

Desde `labs/lab2_mlflow_dvc/`, ejecuta los tres entrenamientos en orden:

```
python src/train.py --model logreg
python src/train.py --model rf
python src/train.py --model xgb
```

Cada uno tarda entre 30 y 60 segundos. Al final de cada uno verás una línea como:

```
==      rf == accuracy=0.8612  f1=0.6877  precision=0.7561  recall=0.6306
roc_auc=0.9120
```

5.4 Paso 4 — Comparar los runs en la UI

Abre `http://localhost:5050` en el navegador. En la barra lateral izquierda, click en el experimento **heart-failure-classifier**. Verás los tres runs.

Para compararlos:

1. Marca las tres casillas a la izquierda de los runs.
2. Pulsa el botón **Compare**.
3. Aparece una vista con cuatro paneles: parámetros, métricas, coordenadas paralelas y dispersión.
4. En el panel de métricas, ordena por F1 descendente. El ganador suele ser **xgb** o **rf**.

5.5 Paso 5 — Registrar el modelo ganador

Tienes dos formas de registrar el modelo: por la UI o por código. Vamos a hacerlo por código, que es lo que harías en un pipeline real.

```
python src/register_best.py --experiment heart-failure-classifier --metric f1
--name heart-failure-clf
```

El script `register_best.py`:

1. Busca el run del experimento `heart-failure-classifier` con mayor F1.
2. Llama a `mlflow.register_model()` con el path del modelo de ese run.
3. Crea una nueva versión del modelo `heart-failure-clf` en el Registry.

Recarga `http://localhost:5050` y entra en la pestaña **Models** arriba a la derecha. Verás `heart-failure-clf` con su versión 1.

5.6 Paso 6 — Promocionar a Staging

En el Registry, las versiones empiezan en stage `None`. Para promocionarla:

```
python src/promote.py --name heart-failure-clf --version 1 --stage Staging
```

Recarga la UI. La versión 1 ahora está en **Staging**.

¿Para qué sirve esto? Cuando en el Lab 3 sirvamos el modelo con FastAPI, la API apuntará a `models:/heart-failure-clf/Staging`. Si mañana quieres cambiar el modelo que sirve en producción, solo tienes que promocionar otro y la API lo recoge en su próxima recarga. **No hay que volver a desplegar código.**

5.7 Paso 7 — Cargar el modelo desde Python

Para terminar, verifiquemos que el modelo es accesible desde Python. Lanza un Python interactivo:

```
python
```

Y ejecuta:

```
import os
import mlflow.pyfunc

os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5050"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

m = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")

print("Run ID:", m.metadata.run_id)
print("Inputs:", len(m.metadata.signature.inputs.inputs), "columnas")
```

Verás algo así:

```
Run ID: 1944ae332b7249a9874dd98fd20c381a
Inputs: 95 columns
```

Esa única línea (`models:/heart-failure-clf/Staging`) es todo lo que necesita la API para cargar el modelo correcto, con su esquema completo y trazabilidad al run que lo entrenó. Eso es el Registry en su esencia.

Sal del Python con `exit()` o `Ctrl+D`.

6. Funciones de MLflow explicadas una a una

Función	Qué hace
<code>mlflow.set_tracking_uri(url)</code>	Indica al cliente Python dónde está el tracking server.
<code>mlflow.set_experiment(name)</code>	Selecciona el experimento. Lo crea si no existe.
<code>mlflow.start_run(run_name=name)</code>	Abre un run nuevo. Se usa como context manager (<code>with</code>).
<code>mlflow.log_param(key, value)</code>	Guarda un único hiperparámetro.
<code>mlflow.log_params(dict)</code>	Guarda varios hiperparámetros a la vez.
<code>mlflow.log_metric(key, value)</code>	Guarda una métrica. Si la llamas varias veces, se guarda como serie temporal.
<code>mlflow.log_metrics(dict)</code>	Igual pero con varias métricas.
<code>mlflow.set_tag(key, value)</code>	Añade un tag al run.
<code>mlflow.set_tags(dict)</code>	Varios tags a la vez.
<code>mlflow.sklearn.log_model(sk_model, artifact_path, signature, input_example, pip_requirements)</code>	Serializa el modelo sklearn, lo sube y registra en el run.
<code>mlflow.pyfunc.load_model(uri)</code>	Carga un modelo desde una URI tipo <code>models:/nombre/Staging</code> o <code>runs:/run_id/model</code> .
<code>mlflow.MlflowClient().register_model(model_uri, name)</code>	Crea una nueva versión de un modelo en el Registry.
<code>mlflow.MlflowClient().transition_model_version_stage(name, version, stage)</code>	Mueve una versión entre stages (Staging, Production, Archived).

7. Métricas de clasificación explicadas

Hemos calculado cinco métricas en cada run. Conviene entender qué mide cada una:

Métrica	Qué mide
Accuracy	Porcentaje de predicciones correctas sobre el total. Engaña en datasets desequilibrados.
Precision	De los que el modelo dijo “positivo”, ¿cuántos lo eran realmente?
Recall	De los que eran “positivo” en la realidad, ¿cuántos detectó el modelo?
F1	Media armónica de precision y recall. Buena métrica general.
ROC-AUC	Capacidad del modelo de ordenar bien (independiente del umbral). Va de 0,5 (aleatorio) a 1,0 (perfecto).

En un problema de detección de fraude o de morosidad te importa **recall** (no perderte casos). En un problema de spam te importa **precision** (no marcar correo legítimo). El F1 es el equilibrio.

8. Checklist de cierre

- ¿Has entrenado tres modelos y aparecen los tres runs en la UI de MLflow?
- ¿Tienes tags `git_commit`, `dataset_version`, `owner` y `framework` en cada run?
- ¿Tienes un modelo logueado con signature en cada run?
- ¿Existe el modelo `heart-failure-clf` versión 1 en el Registry, en stage `Staging`?
- ¿Has podido cargar el modelo desde Python con `models:/heart-failure-clf/Staging`?

Si todo es sí, pasas al módulo 4 (serving).

9. Para profundizar

- **MLflow documentation:** <https://mlflow.org/docs/latest/index.html>. La sección “Tutorial” es muy buena.
- **Designing Machine Learning Systems**, Chip Huyen (O’Reilly, 2022). Capítulo de experiment tracking.
- **MLflow vs Weights & Biases:** comparativa rápida en <https://wandb.ai/site/articles/mlflow-vs-wandb>.

1. Servir un modelo: qué significa y qué patrones existen

Hasta aquí tienes un modelo entrenado, versionado y registrado. Falta llevarlo a producción: que otra aplicación pueda hacerle preguntas y recibir respuestas. Eso se llama **serving**.

No todos los modelos se sirven igual. Hay cuatro patrones clásicos:

Patrón	Cuándo se usa	Ejemplo real
Batch	Predicciones masivas periódicas	Scoring nocturno de una cartera de clientes
Online	Respuesta en milisegundos por petición	Recomendador de productos en una web
Streaming	Datos llegando por una cola	Kafka + consumer que clasifica eventos en tiempo real
Edge	Sin red, en el propio dispositivo	Modelo de reconocimiento de voz en móvil

En este módulo vemos **online**, que es el más común y la base para muchos casos. Concretamente, vamos a servir el modelo `heart-failure-clf` del Lab 2 como una API REST.

2. Las librerías principales: FastAPI, Pydantic, Uvicorn

Estas tres librerías trabajan siempre juntas en Python moderno.

2.1 Qué es FastAPI

FastAPI es un framework para construir APIs REST en Python. Tres ventajas sobre alternativas más antiguas (Flask, Bottle):

1. **Rápido.** Se basa en Starlette, que usa `async/await`, y tiene rendimiento parecido a Node.js o Go.
2. **Validación automática.** Si declaras los tipos de los datos esperados, FastAPI los valida antes de que tu código los reciba.
3. **Documentación automática.** Genera una página interactiva en `/docs` sin que escribas nada extra.

2.2 Qué es Pydantic

Pydantic es la librería que FastAPI usa por debajo para validar datos. Defines una clase que describe el formato esperado del JSON de entrada y Pydantic:

- Convierte automáticamente tipos (string a int si tiene sentido).
- Rechaza datos que no encajan, devolviendo un error 422 al cliente.

Ejemplo de la API del Lab 3 (features clínicas):

```
from pydantic import BaseModel, Field

class Features(BaseModel):
    age: float = Field(ge=18, le=110)
    anaemia: int = Field(ge=0, le=1)
    ejection_fraction: int = Field(ge=5, le=90)
    serum_creatinine: float = Field(ge=0.1, le=20.0)
    sex: int = Field(ge=0, le=1)
    # ... y otras 6 features más
```

`Field(ge=18, le=110)` significa “greater or equal than 18, less or equal than 110”. Si un cliente manda `age=200`, Pydantic devuelve 422 y tu código nunca llega a ver ese dato malo.

2.3 Qué es Uvicorn

Uvicorn es el servidor que ejecuta tu aplicación FastAPI. Es a FastAPI lo que Gunicorn era a Flask: el motor que escucha peticiones HTTP y las pasa a tu código.

Para arrancar tu aplicación:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Lo que dice esto:

- `app.main:app` : importa el módulo `app/main.py` , dentro busca el objeto llamado `app` .
- `--host 0.0.0.0` : escucha en todas las interfaces de red.
- `--port 8000` : en el puerto 8000.

2.4 Instalación de las tres

En el lab del curso van instaladas dentro de la imagen Docker que construyes. Si quieres tenerlas en tu sistema:

```
pip install fastapi==0.115.4 "uvicorn[standard]==0.32.0" pydantic==2.9.2
```

El `[standard]` de uvicorn instala extras útiles (httptools, websockets, watchfiles para autoreload).

3. La otra parte: Docker, en serio

Hasta aquí Docker era una caja negra que arrancabas con `./setup.sh`. En este módulo vamos a construir **nuestra propia imagen**. Es el momento de entender el modelo.

3.1 Imagen vs contenedor

Las dos palabras se confunden todo el rato. La distinción es sencilla:

- **Imagen:** una plantilla congelada. Es como una receta + los ingredientes preparados. No se ejecuta.
- **Contenedor:** una imagen en ejecución. Es el plato cocinado a partir de la receta. Una imagen puede generar muchos contenedores.

3.2 El Dockerfile

Un **Dockerfile** es el fichero que describe cómo construir una imagen. Es una secuencia de instrucciones que parten de otra imagen (la “base”) y van añadiendo capas hasta obtener la imagen final.

Vamos a leer el Dockerfile del Lab 3 línea por línea:

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt
```

Etapas builder:

- `FROM python:3.11-slim AS builder` : parte de una imagen oficial de Python 3.11 en su versión “slim” (sin extras). El alias `builder` permite referirnos a esta etapa luego.
- `WORKDIR /build` : cambia a la carpeta `/build` dentro de la imagen.
- `COPY requirements.txt .` : copia el fichero `requirements.txt` del host al `/build/` de la imagen.
- `RUN pip wheel ...` : compila las dependencias a wheels (binarios precompilados) y los guarda en `/wheels`. Esto se hace en una etapa separada para no arrastrar a la imagen final las herramientas de compilación.

```

FROM python:3.11-slim AS runtime
WORKDIR /app

RUN apt-get update && apt-get install --no-install-recommends -y \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /wheels /wheels
COPY requirements.txt .
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r
requirements.txt \
    && rm -rf /wheels

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
    MODEL_URI=models:/heart-failure-clf/Staging

COPY app/ ./app/

RUN useradd -u 1000 -m svc && chown -R svc /app
USER svc

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \

    CMD curl -fsS http://localhost:8000/health || exit 1

    CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--
workers", "2"]

```

Etapas runtime (la final):

- `FROM python:3.11-slim AS runtime`: parte otra vez de Python slim, pero en una imagen separada.
- `WORKDIR /app`: cambia a `/app`.
- `RUN apt-get ...`: instala `curl` (lo necesitamos para el healthcheck). El `rm -rf` borra los listados de paquetes para reducir el tamaño de la imagen.
- `COPY --from=builder /wheels /wheels`: trae los wheels precompilados desde la etapa builder.
- `RUN pip install ... --find-links=/wheels`: instala las dependencias desde los wheels locales (no descarga nada). Al final borra `/wheels` para ahorrar espacio.
- `ENV ...`: define variables de entorno por defecto. Pueden sobrescribirse al arrancar el contenedor con `-e`.
- `COPY app/ ./app/`: copia el código de la aplicación.
- `RUN useradd ... USER svc`: crea un usuario no privilegiado y se cambia a él. Por seguridad, los contenedores nunca deben correr como root.

- `EXPOSE 8000` : documenta que la aplicación escucha en 8000 (no abre el puerto realmente).
- `HEALTHCHECK ...` : define cómo Docker comprueba que la app está sana. Llama a `/health` cada 30 segundos.
- `CMD [...]` : el comando que se ejecuta cuando arranca el contenedor. Llama a uvicorn con la app.

3.3 Construir la imagen

Desde la carpeta `labs/lab3_serving/` :

```
docker build -t anban/income-api:lab3 .
```

Descomposición:

- `docker build` : instruye a Docker que construya una imagen.
- `-t anban/income-api:lab3` : le pone una etiqueta (nombre:versión).
- `.` : el contexto. Le dice a Docker que use el directorio actual, incluyendo el Dockerfile que hay aquí.

La primera vez tarda 1-3 minutos. Las siguientes, segundos, porque Docker cachea las capas.

3.4 Lanzar un contenedor

```
docker run -d \
  --name anban-income-api \
  --network docker_default \
  -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://anban-minio:9000 \
  -e AWS_ACCESS_KEY_ID=minio \
  -e AWS_SECRET_ACCESS_KEY=minio12345 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:lab3
```

Cada flag:

- `-d` : detached, en segundo plano.
- `--name anban-income-api` : nombre del contenedor, para referirnos a él luego (parar, ver logs, etc.).
- `--network docker_default` : lo metemos en la misma red que el resto del laboratorio. Así puede ver a `anban-mlflow` y `anban-minio` por su nombre.
- `-p 8000:8000` : publica el puerto. El primer 8000 es el del host, el segundo es el del contenedor.
- `-e VAR=value` : define una variable de entorno dentro del contenedor. Sobrescribe los defaults del Dockerfile.
- `anban/income-api:lab3` : la imagen que ejecutamos.

3.5 Comandos útiles para gestionar contenedores

Comando	Qué hace
<code>docker ps</code>	Lista los contenedores en ejecución.
<code>docker ps -a</code>	Lista todos los contenedores (incluso parados).
<code>docker logs anban-income-api</code>	Imprime los logs del contenedor.
<code>docker logs -f anban-income-api</code>	Igual pero “follow”, se queda esperando líneas nuevas.
<code>docker stop anban-income-api</code>	Para el contenedor.
<code>docker start anban-income-api</code>	Lo vuelve a arrancar.
<code>docker restart anban-income-api</code>	Stop + start.
<code>docker rm anban-income-api</code>	Borra el contenedor parado.
<code>docker rm -f anban-income-api</code>	Lo borra incluso si está corriendo.
<code>docker exec -it anban-income-api bash</code>	Entra al contenedor con una shell interactiva.
<code>docker images</code>	Lista las imágenes locales.
<code>docker rmi anban/income-api:lab3</code>	Borra una imagen.
<code>docker system prune -a</code>	Libera espacio borrando imágenes y contenedores no usados.

4. La aplicación: lectura del main.py

El fichero `labs/lab3_serving/app/main.py` define la API. Vamos a leerlo en orden de ejecución.

4.1 Imports y configuración

```
import os, time, uuid
from contextlib import asynccontextmanager
from typing import Any

import mlflow
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse, PlainTextResponse
from pydantic import BaseModel, Field

MODEL_URI = os.environ.get("MODEL_URI", "models:/heart-failure-clf/Staging")

MLFLOW_URI = os.environ.get("MLFLOW_TRACKING_URI", "http://localhost:5000")
```

Lee dos variables de entorno. Si no están, usa defaults. La URI del modelo es lo que conecta esta API con el Model Registry del Lab 2.

4.2 Estado global y schema de entrada

```
state: dict[str, Any] = {
    "model": None,
    "run_id": None,
    "signature": None,
    "loaded_at": None,
    "predictions": 0,
    "errors": 0,
    "latency_sum_ms": 0.0,
}

class Features(BaseModel):
    age: float = Field(ge=18, le=110, description="Edad en años")
    anaemia: int = Field(ge=0, le=1, description="1 si hay anemia")
    creatinine_phosphokinase: int = Field(ge=1, le=12000,
        description="CPK en suero (U/L)")
    diabetes: int = Field(ge=0, le=1, description="1 si es diabético")
    ejection_fraction: int = Field(ge=5, le=90,
        description="Fracción de eyección ventricular (%)")
    high_blood_pressure: int = Field(ge=0, le=1)
    platelets: float = Field(ge=10000, le=1000000)
    serum_creatinine: float = Field(ge=0.1, le=20.0)
    serum_sodium: int = Field(ge=100, le=160)
    sex: int = Field(ge=0, le=1, description="0=mujer, 1=hombre")
    smoking: int = Field(ge=0, le=1)
```

- `state` es un diccionario donde guardamos el modelo cargado y contadores para métricas.
- `Features` es la clase Pydantic que describe el JSON de entrada. Cada campo lleva sus restricciones. FastAPI las usa para validar cada petición automáticamente.

4.3 Lifespan: cargar el modelo al arrancar

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    mlflow.set_tracking_uri(MLFLOW_URI)
    state["model"] = mlflow.pyfunc.load_model(MODEL_URI)
    md = state["model"].metadata
    state["run_id"] = md.run_id if md else None
    state["signature"] = md.signature if md else None
    state["loaded_at"] = time.time()
    yield
```

El **lifespan** es un hook de FastAPI que se ejecuta una vez al arrancar la app y otra al pararla. Aquí lo usamos para cargar el modelo una sola vez. Sin esto, lo cargarías en cada petición y la API sería 100 veces más lenta.

4.4 Endpoints

```
@app.get("/health")
def health():
    return {"status": "ok" if state["model"] is not None else "loading"}
```

`/health` lo usaría un load balancer para saber si esta réplica está sana. Si la app aún no ha cargado el modelo, devuelve “loading”.

```
@app.get("/version")
def version():
    return {
        "model_uri": MODEL_URI,
        "run_id": state["run_id"],
        "signature": str(state["signature"]),
    }
```

`/version` te dice qué modelo está sirviendo. Útil para auditoría rápida y para depurar despliegues.

```
@app.post("/predict", response_model=Prediction)
def predict(payload: Features) -> Prediction:
    rid = uuid.uuid4().hex[:12]
    t0 = time.perf_counter()
    X = _to_frame(payload)
    proba = float(state["model"].predict(X)[0])
    label = int(proba >= 0.5)
    dt = (time.perf_counter() - t0) * 1000
    return Prediction(label=label, proba=proba, ...)
```

`/predict` es el endpoint principal:

1. Pydantic ya validó el payload antes de llegar aquí. Si era inválido, FastAPI devolvió 422 sin tocar este código.
2. Genera un `request_id` único.
3. Convierte el payload a un DataFrame con el formato que espera el modelo (one-hot encoding).
4. Llama a `predict()` del modelo.
5. Aplica un umbral (0,5) para convertir probabilidad en etiqueta.
6. Devuelve la respuesta estructurada.

```
@app.get("/metrics", response_class=PlainTextResponse)
def metrics() -> str:
    return (
        f"# HELP api_predictions_total total predictions served\n"
        f"# TYPE api_predictions_total counter\n"
        f"api_predictions_total {state['predictions']}\n"
        ...
    )
```

`/metrics` devuelve métricas en formato Prometheus. Cualquier Prometheus puede hacer scraping de este endpoint y graficar las métricas en Grafana.

5. Antes de la práctica: requisitos

Para hacer este lab:

1. **Lab 2 completado:** tienes el modelo `heart-failure-clf` versión 1 en stage `Staging`.
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab3_serving/`.**

6. Ejercicio: empaquetar y servir el modelo

Objetivo del ejercicio: construir una imagen Docker con tu API, lanzarla, probar todos los endpoints con curl y comprobar que la validación Pydantic rechaza datos inválidos.

6.1 Paso 1 — Construir la imagen

```
docker build -t anban/income-api:lab3 .
```

Mientras compila, ves la lista de pasos del Dockerfile. La primera vez tarda 1-3 minutos. La imagen final pesa unos 1,2 GB (es grande porque incluye PyTorch y xgboost; en producción real usarías una imagen base más pequeña).

Verifica:

```
docker images anban/income-api
```

6.2 Paso 2 — Lanzar el contenedor

Antes de lanzar, asegúrate de que no haya un contenedor antiguo con el mismo nombre:

```
docker rm -f anban-income-api 2>/dev/null
```

Y lanzas:

```
docker run -d \
  --name anban-income-api \
  --network docker_default \
  -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://anban-minio:9000 \
  -e AWS_ACCESS_KEY_ID=minio \
  -e AWS_SECRET_ACCESS_KEY=minio12345 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:lab3
```

Espera unos 20-30 segundos a que arranque (tiene que cargar el modelo desde MinIO).

Verifica que está corriendo:

```
docker ps | grep anban-income-api
```

6.3 Paso 3 — Probar /health

```
curl -s http://localhost:8000/health
```

Salida esperada:

```
{"status": "ok", "uptime_s": 15}
```

Si todavía dice "loading", espera 10 segundos más. El modelo está descargándose desde MinIO.

6.4 Paso 4 — Probar /version

```
curl -s http://localhost:8000/version
```

Salida:

```
{
  "model_uri": "models:/heart-failure-clf/Staging",
  "run_id": "1944ae332b7249a9874dd98fd20c381a",
  "signature": "..."}

```

Anota el `run_id`: corresponde al run del Lab 2 que entrenó al ganador. Esa es la trazabilidad completa.

6.5 Paso 5 — Hacer una predicción real

```
curl -s -X POST http://localhost:8000/predict \
-H "content-type: application/json" \
-d '{
  "age": 75,
  "anaemia": 0,
  "creatinine_phosphokinase": 582,
  "diabetes": 0,
  "ejection_fraction": 20,
  "high_blood_pressure": 1,
  "platelets": 265000,
  "serum_creatinine": 1.9,
  "serum_sodium": 130,
  "sex": 1,
  "smoking": 0
}'
```

Estos son datos reales del primer paciente del dataset. Edad 75, fracción de eyección muy baja (20 %, lo normal es 55-70 %), creatinina alta (1,9 mg/dL, lo normal es < 1,3). Es un caso clínico de alto riesgo.

Respuesta:

```
{
  "label": 1,
  "proba": 1.0,
  "model_uri": "models:/heart-failure-clf/Staging",
  "request_id": "a3f8c2d4e9b0",
  "latency_ms": 4.21
}
```

`label=0` significa “no gana más de 50K”. `proba` es la probabilidad que el modelo asigna a la clase positiva. `latency_ms` mide cuánto tardó la inferencia.

6.6 Paso 6 — Probar la validación con un dato malo

```
curl -s -X POST http://localhost:8000/predict \
-H "content-type: application/json" \
-d '{
  "age": 200,
  "anaemia": 0,
  "creatinine_phosphokinase": 100,
  "diabetes": 0,
  "ejection_fraction": 35,
  "high_blood_pressure": 0,
  "platelets": 250000,
  "serum_creatinine": 1.0,
  "serum_sodium": 140,
  "sex": 1,
  "smoking": 0
}'
```

Pydantic debería rechazar el `age=200` con un 422:

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": ["body", "age"],
      "msg": "Input should be less than or equal to 110",
      "input": 200
    }
  ]
}
```

El modelo no se llega a ejecutar: la validación lo paró antes.

6.7 Paso 7 — Probar el endpoint de métricas

```
curl -s http://localhost:8000/metrics
```

Salida en formato Prometheus:

```
# HELP api_predictions_total total predictions served
# TYPE api_predictions_total counter
api_predictions_total 1
# HELP api_errors_total total errors
# TYPE api_errors_total counter
api_errors_total 0
# HELP api_latency_avg_ms average inference latency
# TYPE api_latency_avg_ms gauge
api_latency_avg_ms 4.21
```

6.8 Paso 8 — Probar la documentación automática

Abre <http://localhost:8000/docs> en el navegador. Verás la documentación Swagger generada automáticamente por FastAPI. Puedes hacer las pruebas anteriores desde el navegador con el botón “Try it out”.

6.9 Paso 9 — Ver los logs

```
docker logs anban-income-api
```

Verás líneas como:

```
INFO:      Started server process [7]
INFO:      Waiting for application startup.
loading model from models:/heart-failure-clf/Staging
model loaded · run_id=...
INFO:      Application startup complete.
INFO:      172.23.0.1:51920 - "POST /predict HTTP/1.1" 200 OK
INFO:      172.23.0.1:51924 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
```

6.10 Paso 10 — Detener el contenedor

Cuando termines el lab:

```
docker stop anban-income-api
```

El contenedor se queda parado pero no se borra. Para volver a arrancarlo: `docker start anban-income-api`.

7. Conceptos de performance

Cuando despliegues una API real en producción, te van a hacer preguntas sobre rendimiento. Conviene tener el vocabulario claro.

Métrica	Qué significa
Latencia p50	La mitad de las peticiones tardan menos que esto.
Latencia p95	El 95 % tardan menos. Métrica habitual de SLA.
Latencia p99	El peor 1 % queda por debajo. Importante en escala.
Throughput (RPS)	Peticiones por segundo que aguanta sin perder calidad.
Cold start	Tiempo desde frío hasta la primera respuesta.

No te quedes con la media: una API con media de 100 ms y p99 de 5 s no es aceptable. Siempre mira la cola.

8. Checklist de cierre

- ¿Has construido la imagen `anban/income-api:lab3` ?
- ¿El contenedor está corriendo y responde a `/health` con `ok` ?
- ¿ `/version` devuelve el `run_id` correcto?
- ¿ `/predict` devuelve una respuesta estructurada con `label`, `proba`, `request_id` y `latency_ms`?
- ¿La validación Pydantic ha devuelto 422 con `age=200` ?
- ¿Has visto las métricas en formato Prometheus en `/metrics` ?

Si todo es sí, pasas al módulo 5 (CI/CD y drift).

9. Para profundizar

- **FastAPI documentation:** <https://fastapi.tiangolo.com>. La sección “Tutorial - User Guide” es excelente.
- **Pydantic v2:** <https://docs.pydantic.dev>. Para validaciones más complejas.
- **Docker docs:** <https://docs.docker.com>. El “Get Started” cubre todo lo que hemos visto.
- **BentoML:** <https://docs.bentoml.org>. Alternativa especializada en servir modelos ML, con batching dinámico y otras optimizaciones.

1. Las tres siglas: CI, CD y CT

En MLOps se usan tres siglas que suelen confundirse:

- **CI (Continuous Integration)**. Cada commit pasa por test, lint y build automáticos. Garantiza que el repositorio no se rompe.
- **CD (Continuous Delivery o Continuous Deployment)**. La salida del CI se puede desplegar a producción con un clic (delivery) o automáticamente (deployment).
- **CT (Continuous Training)**. Específico de ML. Reentrenamiento automático del modelo cuando llegan datos nuevos o cuando se detecta drift.

En un sistema ML maduro corren los tres bucles:

```
commit → CI → test + train + eval + register
                                     ↓
datos nuevos → CT → reentrena → eval vs prod
                                     ↓
                        schedule → CD → despliega si mejora
```

El de la izquierda es el de DevOps. Los otros dos son los que añade MLOps.

2. Estrategias de despliegue

Cuando promocionas un modelo nuevo a producción, hay tres estrategias clásicas para hacerlo sin riesgo:

Estrategia	Cómo funciona	Cuándo elegirla
Blue-green	Dos entornos paralelos; cambias el tráfico de uno a otro	Crítico, necesitas rollback en segundos
Canary	Empiezas con 5 % del tráfico, vas subiendo hasta 100 %	Detectar bugs sin daño masivo
Shadow	El modelo nuevo recibe el tráfico real pero no responde al cliente; comparas con el viejo	Validar el nuevo durante días antes de cambiar

En ML, la estrategia **shadow** es valiosísima: te permite ejecutar el modelo nuevo durante una semana con tráfico real antes de sustituir nada.

3. Monitorización: dos familias de métricas

Cuando un modelo está en producción, tienes que medirlo en dos planos distintos:

3.1 Métricas de servicio

Son las de cualquier API:

- Latencia (p50, p95, p99).
- Throughput (peticiones por segundo).
- Tasa de errores (5xx, 4xx).
- Uso de CPU y memoria del contenedor.

Estas las recoges con Prometheus + Grafana (o con servicios cloud equivalentes como CloudWatch, Data-dog, New Relic).

3.2 Métricas de modelo

Son específicas de ML:

- **Distribución de outputs.** Si el modelo ahora dice “sí” un 80 % de las veces en lugar del 20 % habitual, algo ha cambiado.
- **Accuracy proxy.** Cuando tardas en tener la etiqueta real (ej. “morosidad a 90 días”), usas un proxy hasta que llegue.
- **Drift de inputs.** La distribución de los datos que recibe el modelo cambia respecto a los datos con los que entrenó. Es el tema del próximo apartado.

4. Drift: la enfermedad silenciosa

Un modelo en producción puede dejar de funcionar **sin que se rompa nada técnico**. El código no falla, la API responde, las métricas de servicio están verdes. Pero las predicciones son cada vez peores. Eso es **drift**: la realidad ha cambiado y el modelo ya no la representa.

Hay tres tipos:

Tipo	Qué cambia	Ejemplo
Data drift (covariate shift)	$P(X)$, la distribución de inputs	Tras la pandemia, la edad media de los clientes online subió 10 años
Concept drift	$P(y X)$, la relación entre input y output	Lo que antes predecía morosidad ya no lo predice
Label drift	$P(y)$, la distribución de la etiqueta	Antes 20 % de morosos, ahora 35 %

4.1 Tests estadísticos para detectar drift

Cuatro tests habituales:

Test	Tipo de variable	Comentario
PSI	Numérica	Umbrales intuitivos: 0,1 estable, 0,1-0,25 vigilar, >0,25 drift
Kolmogorov-Smirnov (KS)	Numérica	Compara distribuciones, devuelve p-value
Chi-cuadrado	Categorica	Mide independencia entre dos categóricas
Wasserstein	Numérica	Distancia “earth mover” entre distribuciones

4.2 PSI explicado a fondo

PSI (Population Stability Index) es el más usado en industria por su simplicidad. Para una variable numérica el algoritmo es:

1. Divides la variable en N bins (típicamente 10 deciles del dataset de referencia).
2. Para cada bin, calculas la proporción de registros del dataset de referencia (`expected_i`) y del dataset actual (`actual_i`).

3. Sumas sobre todos los bins:

$$\text{PSI} = \sum_i (\text{actual}_i - \text{expected}_i) * \ln(\text{actual}_i / \text{expected}_i)$$

Interpretación operativa:

Valor de PSI	Significado
< 0,1	Distribución estable, sin acción
0,1 a 0,25	Cambio moderado, vigilar
≥ 0,25	Drift significativo, investigar y posiblemente reentrenar

5. Las librerías principales

5.1 Evidently

Evidently es la librería de código abierto más usada para monitorizar modelos. Te permite generar informes HTML interactivos comparando dos datasets (referencia vs actual) y detectar drift, calidad de datos y rendimiento.

Para instalarla:

```
pip install evidently
```

Es opcional: si no está instalada, nuestro script `drift_report.py` cae a un fallback que calcula PSI a mano y genera un HTML mínimo.

5.2 Numpy

NumPy es la base del cálculo numérico en Python. La usamos para crear histogramas y calcular cuantiles.

```
pip install numpy
```

Operaciones que aparecen en el script:

```
import numpy as np

# Cuantiles (para crear bins):
breaks = np.quantile(reference.dropna().to_numpy(), np.linspace(0, 1, 11)
)

# Histograma:
hist, _ = np.histogram(current.dropna(), bins=breaks)

# Operaciones elemento a elemento:
psi = np.sum((actual_pct - expected_pct) * np.log(actual_pct / expected_pct))
```

5.3 GitHub Actions

GitHub Actions no es una librería, es un servicio integrado en GitHub. Permite ejecutar workflows (pipelines de CI/CD) cada vez que ocurre un evento en tu repositorio (push, pull request, schedule, manual).

Un workflow es un fichero YAML en `.github/workflows/`. Ejemplo mínimo:

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest
```

6. Antes de la práctica: requisitos

Para hacer este lab necesitas:

1. **Lab 1 y Lab 2 completados.** El Lab 4 lee el test.parquet del Lab 1 y el modelo registrado del Lab 2.
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab4_cicd_monitoring/`.**

7. Ejercicio: detectar drift y promoción condicional

Objetivo del ejercicio: generar un dataset con drift inyectado, calcular el PSI por feature, ver el reporte HTML, y probar el mecanismo de promoción condicional que solo asciende un modelo si bate al actual de Production.

7.1 Paso 1 — Exportar variables de entorno

Las mismas que en el Lab 2:

```
export MLFLOW_TRACKING_URI=http://localhost:5050
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

7.2 Paso 2 — Generar un dataset con drift sintético

El script `synthetic/make_drift.py` toma el `test.parquet` del Lab 1 y le aplica cuatro modificaciones que simulan cambios clínicos plausibles:

- Suma 8 años a `age` (la población de pacientes envejece).
- Aumenta la prevalencia de `high_blood_pressure` en 10 %.
- Multiplica `creatinine_phosphokinase` por 1,5 (cambio de método de laboratorio).
- Cambia aleatoriamente el 4 % de las etiquetas `DEATH_EVENT` (leve cambio de prevalencia).

Ejecútalo:

```
mkdir -p data/processed reports
python synthetic/make_drift.py
```

Salida:

```
OK · data/processed/drifted.parquet (60 filas con drift inyectado)
```

7.3 Paso 3 — Generar el reporte de drift

El script `src/drift_report.py` compara dos parquet (referencia vs actual) y genera un HTML con el análisis. Lo usamos así:

```
python src/drift_report.py \  
  --reference ../lab1_dataops/data/processed/test.parquet \  
  --current data/processed/drifted.parquet \  
  --output reports/drift.html
```

Si Evidently está instalado, hace un análisis muy completo (gráficos de distribuciones, mapas de correlaciones, etc.). Si no, cae al fallback PSI que produce un HTML simple con una tabla.

Cuando termina, abre `reports/drift.html` en el navegador.

7.4 Paso 4 — Interpretar los resultados

Mira `reports/drift.json`. Es un diccionario `{feature: PSI}`.

```
cat reports/drift.json | head -20
```

Las features con `PSI > 0,25` son las que han cambiado mucho. Típicamente: `age`, `creatinine_phosphokinase`, `high_blood_pressure`.

Reflexión: si vieras este reporte en producción real, ¿qué harías? Tres opciones razonables:

- Reentrenar el modelo con datos recientes.
- Investigar de dónde viene el cambio (¿un bug en el ETL?, ¿cambio de mercado?, ¿error en la fuente?).
- Aceptar el drift si es esperado y no afecta a la métrica de negocio.

7.5 Paso 5 — La lógica de promoción condicional

El script `src/promote_if_better.py` automatiza la decisión: solo promociona un candidato a Producción si bate al actual en una métrica concreta, por un margen mínimo.

Léelo:

```
cat src/promote_if_better.py
```

La lógica clave:

```

candidate_metric = metric_for_version(client, name, candidate, args.metric)

prod_metric = metric_for_version(client, name, prod, args.metric)

threshold = prod_metric * (1 + args.min_improvement)
if candidate_metric >= threshold:
    client.transition_model_version_stage(name, candidate, "Production")

    print("PROMOTE")
else:
    print("REJECT")

```

El parámetro `--min-improvement 0.01` significa “solo promociona si mejora al menos un 1 %”. Esto evita que el sistema oscile entre modelos muy parecidos.

7.6 Paso 6 — Probarlo en local

Si aún no hay versión en Production, el script promociona la candidata automáticamente. Pruébalo:

```

python src/promote_if_better.py \
  --name heart-failure-clf \
  --metric f1 \
  --min-improvement 0.01

```

Salida típica:

```

candidate v1: f1=0.6877
no hay Production aún · promovemos candidato a Staging

```

7.7 Paso 7 — Revisar el workflow de GitHub Actions

El fichero `.github/workflows/ml.yml` define un workflow que en un repositorio real se ejecutaría en cada push. No vamos a ejecutarlo aquí (necesitas GitHub), pero conviene leerlo:

```

cat .github/workflows/ml.yml

```

Estructura habitual:

```

name: ml-pipeline
on:
  push:
    branches: [main]
  pull_request:
  schedule:
    - cron: "0 3 * * 0" # los domingos a las 3 AM
  workflow_dispatch: # botón "Run workflow" en la UI

jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps: ...

  train:
    needs: lint-test
    runs-on: ubuntu-latest
    steps: ...

  evaluate-and-promote:
    needs: train
    runs-on: ubuntu-latest
    steps: ...

  drift-report:
    schedule: "0 3 * * 0"
    steps: ...

```

Tres jobs encadenados con `needs:` (cada uno espera al anterior). El último corre solo en schedule (semanal) y genera el reporte de drift.

8. Checklist de cierre

- ¿Has generado el dataset `drifted.parquet` ?
- ¿Has generado `reports/drift.html` y lo has abierto en el navegador?
- ¿Has identificado al menos tres features con $PSI > 0,25$?
- ¿Has probado `promote_if_better.py` y has visto la decisión PROMOTE o REJECT?
- ¿Has leído el workflow `.github/workflows/ml.yml` y entiendes los jobs encadenados?

Si todo es sí, pasas al módulo 6 (governance y caso end-to-end).

9. Para profundizar

- **Evidently**: <https://docs.evidentlyai.com>. Documentación con ejemplos visuales.
- **GitHub Actions docs**: <https://docs.github.com/actions>.
- **Reliable Machine Learning**, Cathy Chen et al. (O'Reilly). Capítulos de monitoring.
- **NannyML**: <https://nannyml.readthedocs.io>. Estima accuracy cuando aún no tienes las etiquetas reales.

1. Más allá del MVP: gobernanza del modelo

Hasta aquí has visto cómo construir un sistema MLOps funcional. Falta un capítulo que casi siempre se olvida: la **gobernanza**. Es lo que diferencia un proyecto que vive seis meses de uno que vive seis años.

La gobernanza cubre cinco bloques:

1. Documentación del modelo (Model Card).
2. Trazabilidad y auditoría.
3. Cumplimiento legal (RGPD, AI Act).
4. Sesgo, equidad y subgrupos.
5. Reproducibilidad estricta.

2. La Model Card

Una **Model Card** es un documento estandarizado que describe un modelo. Lo propusieron Mitchell et al. en Google en 2019 y se ha convertido en un estándar de facto.

2.1 Qué debe contener

Sección	Contenido
Propósito	Para qué se entrena y se sirve el modelo
Datos	Origen, periodo, tamaño, sesgos conocidos del dataset
Métricas globales	Accuracy, F1, etc., en test
Métricas por subgrupo	F1 por sexo, edad, raza, geografía
Limitaciones	Casos en los que el modelo no funciona bien
Contraindicaciones	Casos en los que NO se debe usar
Mantenedor	Quién es responsable
Versionado	Hash de commit, versión, fecha

Sin Model Card, dentro de un año nadie sabrá qué hacía ese modelo.

2.2 Generarla automáticamente

Lo que **no se debe hacer**: redactar la Model Card a mano cada vez. Se olvida, se desactualiza, queda obsoleta.

Lo que **sí se debe hacer**: generar las partes automatizables a partir del Model Registry (métricas, run_id, commit) y rellenar a mano solo las partes que requieren criterio humano (limitaciones, contraindicaciones, decisiones de gobernanza).

En el Lab 5 verás un script que hace exactamente esto.

3. Audit trail

Audit trail es el registro completo de qué predicciones hizo cada modelo, cuándo, con qué inputs. En sectores regulados (banca, salud, seguros, justicia) es obligatorio. En cualquier sector es útil para depurar.

Mínimo viable de un audit trail:

- Cada petición a `/predict` se loguea con:
 - `request_id` único.
 - Timestamp ISO 8601.
 - Versión del modelo que la atendió.
 - Inputs (puede ser resumido o hasheado si hay PII).
 - Output (label + proba).
- Los logs se mandan a un sistema centralizado (ELK, Splunk, CloudWatch Logs, etc.).
- Se conservan según regulación local (en banca europea, mínimo cinco años).

En la API del Lab 3 ya guardamos `request_id` y la respuesta lleva `model_uri` y `latency_ms`. Falta enviarlo a un log centralizado.

4. RGPD y AI Act

4.1 RGPD (Reglamento General de Protección de Datos)

Aplica en toda la Unión Europea desde 2018. El artículo más relevante para ML es el **Artículo 22**:

“El interesado tendrá derecho a no ser objeto de una decisión basada únicamente en el tratamiento automatizado, incluida la elaboración de perfiles, que produzca efectos jurídicos en él o le afecte significativamente.”

Si tu modelo toma decisiones automatizadas que afectan a personas (denegar crédito, contratar, asignar becas), el afectado tiene derecho a:

- 1. **Intervención humana** en la decisión.
- 2. **Expresar su punto de vista**.
- 3. **Impugnar la decisión**.

4.2 AI Act europeo

Entró en vigor escalonado en 2024-2026. Clasifica los sistemas de IA por nivel de riesgo:

Nivel de riesgo	Qué incluye	Obligaciones
Inaceptable	Sistemas prohibidos (puntuación social tipo China)	No se permiten
Alto	Scoring crediticio, RRHH, justicia, dispositivos médicos	Documentación, transparencia, registro, supervisión humana
Limitado	Chatbots, deepfakes	Etiquetado obligatorio
Mínimo	Filtros antispam, recomendadores triviales	Sin obligaciones específicas

Si tu modelo es de “alto riesgo”, la Model Card no es opcional: forma parte de la documentación obligatoria.

5. Sesgo y fairness

Una métrica global esconde disparidades. Calcula siempre:

- F1, precision y recall **por subgrupo** (sex, race, edad).
- Tasa de falsos positivos por subgrupo.
- Disparate impact (proporción de decisiones positivas entre grupos).

Si encuentras disparidad grande, tienes cuatro decisiones posibles:

1. **Reentrenar con técnicas de fairness** (reweighing, adversarial debiasing, post-processing).
2. **Cambiar el umbral por subgrupo.**
3. **Documentar la disparidad y aceptarla** (con justificación transparente).
4. **No desplegar el modelo.**

5.1 Decisión humana

Para casos críticos, **siempre** debe haber un humano en el bucle:

- Denegación de crédito.
- Diagnóstico médico.
- Selección de personal.
- Reconocimiento facial en espacios públicos.

El modelo recomienda, un humano decide.

6. Reproducibilidad estricta

Para reconstruir un modelo dentro de un año (por ejemplo, porque un regulador lo pide), necesitas conservar **cinco cosas**:

Componente	Cómo se versiona
Código	git commit hash
Datos	DVC hash MD5
Parámetros	params.yaml o MLflow params
Entorno	requirements.txt + Python version + Docker image hash
Seed aleatoria	random_state fijado en cada librería estocástica

Si tienes los cinco, eres reproducible. Si te falta uno solo, no.

7. Costes y FinOps en ML

Algunos órdenes de magnitud aproximados (varían según cloud):

- Entrenamiento BERT base en GPU cloud: 50 a 200 EUR.
- Inferencia en CPU vs GPU: hasta 100x diferencia de coste para modelos pequeños.
- Endpoint dedicado 24/7 vs serverless: hasta 10x diferencia para cargas bajas.

Preguntas que tu equipo debería responder antes de elegir arquitectura:

1. ¿Online o batch? Si toleras 1 hora de latencia, batch es 10x más barato.
2. ¿GPU o CPU? Para modelos pequeños o medianos, CPU sobra.
3. ¿Autoscaling? Si tu tráfico es 9-18h, no pagues 24/7.
4. ¿Spot instances? Hasta 70 % de descuento para cargas tolerantes a interrupción.

8. LLMOps en un vistazo

Si trabajas con modelos de lenguaje, aplica todo lo anterior más unos cuantos términos nuevos:

Término	Significado
Prompt versioning	Cada prompt es código y se versiona como tal
Eval	Medir la salida de un LLM es difícil. Herramientas: Promptfoo, LangSmith, Ragas
Guardrails	Validar inputs y outputs (Guardrails AI, NeMo Guardrails)
RAG (Retrieval-Augmented Generation)	Conectar el LLM a tu base documental
Vector DB	Pinecone, Weaviate, pgvector, Qdrant

No los tocamos en este curso. Si te interesa profundizar, hay un curso aparte específico de LLMOps.

9. Antes de la práctica: requisitos

Para hacer el ejercicio final del curso necesitas:

1. **Los labs 1, 2, 3 y 4 completados.**
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab5_e2e/`.**

10. Ejercicio final: recorrido end-to-end + Model Card

Objetivo del ejercicio: verificar que tu pipeline MLOps funciona de principio a fin, generar la Model Card del modelo en Staging y rellenar a mano las secciones que requieren juicio humano.

10.1 Paso 1 — Comprobar que todas las piezas están vivas

Antes de ejecutar nada, comprueba el estado del laboratorio:

```
./doctor.sh
```

Tiene que terminar con “Todo en orden”. Si no, arregla lo que diga antes de seguir.

10.2 Paso 2 — Verificar la cadena de artefactos

Comprueba que cada pieza de los labs anteriores está en su sitio:

Lab 1 — Datos:

```
ls -lh ../lab1_dataops/data/processed/
```

Debes ver `train.parquet` y `test.parquet`.

Lab 2 — Modelo en Registry:

```
curl -s 'http://localhost:5050/api/2.0/mlflow/registered-models/get?name=heart-failure-clf' | head -20
```

Debes ver el modelo `heart-failure-clf` con al menos una versión en stage `Staging` o `Production`.

Lab 3 — API (opcional, si quieres tenerla viva):

```
curl -s http://localhost:8000/health
```

Si la API no está arriba, no pasa nada para este lab.

Lab 4 — Reporte de drift:

```
ls -lh ../lab4_cicd_monitoring/reports/
```

Debes ver `drift.html` y `drift.json`.

10.3 Paso 3 — Generar la Model Card automáticamente

El script `run.sh` del Lab 5 genera la Model Card a partir del Registry. Las secciones que se generan solas:

- Propósito (texto fijo).
- Datos (texto fijo).
- Algoritmo y framework (extraído de los tags del run).
- Hiperparámetros (extraídos de los params).
- Métricas globales (extraídas de las metrics).
- Run ID y commit de git (extraídos de los tags).

Lo que tendrás que rellenar tú a mano:

- Métricas por subgrupo (F1 por sex, race).
- Decisiones de gobernanza.
- Limitaciones y contraindicaciones específicas a tu caso.

Ejecuta el script asistido:

```
./run.sh
```

Al terminar, mira el fichero generado:

```
cat MODEL_CARD.md
```

10.4 Paso 4 — Calcular métricas por subgrupo (ejercicio práctico)

Esto va a tu cuenta. Carga el modelo y el test set, calcula F1 por sexo. Crea un fichero `subgroup_metrics.py` con este contenido:

```

import os
import mlflow.pyfunc
import pandas as pd
from sklearn.metrics import f1_score

# Configurar MLflow
os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5050"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

# Cargar el modelo desde el Registry
model = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")

# Cargar el test set
test = pd.read_parquet("../lab1_dataops/data/processed/test.parquet")
y_true = test["DEATH_EVENT"]
X = test.drop(columns=["DEATH_EVENT"])

# Predicciones globales
y_pred = (model.predict(X) >= 0.5).astype(int)
print(f"F1 global: {f1_score(y_true, y_pred):.4f}")

# F1 por sexo (en este dataset, 0=mujer, 1=hombre)
for sex_value, label in [(1, "Hombre"), (0, "Mujer")]:
    mask = (X["sex"] == sex_value)
    if mask.sum() > 0:
        f1 = f1_score(y_true[mask], y_pred[mask])
        print(f"F1 sex={label} (n={mask.sum()}) : {f1:.4f}")

# F1 por edad (mayores/menores de 65)
for label, mask in [(">=65 años", X["age"] >= 65), ("<65 años", X["age"] < 65)]:
    if mask.sum() > 0:
        f1 = f1_score(y_true[mask], y_pred[mask])
        print(f"F1 {label} (n={mask.sum()}) : {f1:.4f}")

```

Ejecútalo:

```
python subgroup_metrics.py
```

Observa la diferencia. En este dataset, típicamente:

```

F1 global: 0.5455
F1 sex=Hombre (n=39): 0.5217
F1 sex=Mujer (n=21): 0.5714
F1 ≥65 años (n=33): 0.6087
F1 <65 años (n=27): 0.4286

```

Aquí vemos algo interesante: el modelo predice peor para los pacientes **jóvenes**. Es esperable porque los eventos de muerte por insuficiencia cardíaca son menos frecuentes en menores de 65, así que el modelo tiene menos casos positivos en los que aprender.

Pregúntate: si tu modelo real tuviera esta disparidad, ¿lo desplegarías? ¿Qué harías para mitigarla? Anótalo en la Model Card, sección “Limitaciones”.

10.5 Paso 5 — Rellenar las secciones manuales

Abre `MODEL_CARD.md` en tu editor. Rellena:

- **Contraindicaciones:** marca con `[x]` los usos en los que NO se debe emplear este modelo (los cuatro que vienen por defecto son buenos: scoring crediticio real, selección de personal, decisiones automatizadas RGPD Art. 22, producción real con tráfico).
- **Métricas por subgrupo:** añade las F1 que acabas de calcular.
- **Governance:** marca las casillas que apliquen y añade notas.

Guarda. La Model Card está completa.

10.6 Paso 6 — Checklist final del curso

Asegúrate de marcar lo que has completado:

DataOps

- ☐ Repositorio Git inicializado en `lab1_dataops`.
- ☐ Dataset versionado por DVC, no por Git.
- ☐ Pipeline reproducible (`dvc.yaml`).
- ☐ `train.parquet` y `test.parquet` generados.

Tracking y Registry

- ☐ Experimento con 3 runs en MLflow.
- ☐ Modelo `heart-failure-clf` registrado.
- ☐ Versión 1 en stage Staging.

Serving

- ☐ Imagen Docker construida.
- ☐ API respondiendo a `/health`, `/version`, `/predict`, `/metrics`.
- ☐ Pydantic rechaza entradas inválidas con 422.

CI/CD y drift

- ☐ `reports/drift.html` con resultados visibles.
- ☐ `promote_if_better.py` ejecutado al menos una vez.

Governance

- ☐ `MODEL_CARD.md` generada y completada a mano.
- ☐ F1 por subgrupos calculado.

Si tienes todo marcado, **has completado el curso.**

11. Hoja de ruta de aprendizaje

Nivel	Skills
Junior MLOps	Docker, Git, Python, MLflow básico, FastAPI, CI básico
Mid	Kubernetes, Terraform, una feature store, Prometheus + Grafana + alerting, un orquestador (Airflow o Prefect)
Senior	Diseño de plataforma ML completa, multi-cloud, optimización de costes, governance + cumplimiento, multitenancy, edge ML, LLMOps

Pasar de junior a mid suele llevar entre uno y dos años de práctica seria. De mid a senior, otros dos o tres años.

12. Bibliografía y recursos

Libros

- *Designing Machine Learning Systems*, Chip Huyen (O'Reilly, 2022). El más completo y actual.
- *Practical MLOps*, Noah Gift (O'Reilly, 2021). Muy hands-on.
- *Machine Learning Engineering*, Andriy Burkov. Conciso y excelente.
- *Reliable Machine Learning*, Cathy Chen et al. (O'Reilly). SRE aplicado a ML.

Webs

- **ml-ops.org**: hoja de ruta de la comunidad.
- **MLOps Community**: Slack muy activo y blog.
- **Made With ML** (<https://madewithml.com>): tutoriales gratuitos.

Whitepapers fundacionales

- Sculley et al., *Hidden Technical Debt in ML Systems* (2015).
- Google, *MLOps: Continuous delivery and automation pipelines in ML*.
- Mitchell et al., *Model Cards for Model Reporting* (2019).

Esto es una chuleta de uso. Toda la información operativa que necesitas durante los dos días en un solo sitio. Imprime esta hoja si lo prefieres.

Stack del curso

```
# Levantar
docker compose -f docker/docker-compose.yml up -d

# Ver estado
docker compose -f docker/docker-compose.yml ps

# Logs de un servicio
docker compose -f docker/docker-compose.yml logs -f mlflow

# Parar todo
docker compose -f docker/docker-compose.yml down

# Limpiar volúmenes (cuidado, borra datos)
docker compose -f docker/docker-compose.yml down -v
```

URLs y credenciales

Servicio	URL	Usuario / Pass
MLflow Tracking	http://localhost:5000	—
MinIO Console	http://localhost:9001	minio / minio12345
MinIO API	http://localhost:9000	minio / minio12345
Postgres	localhost:5432	mlflow / mlflow
JupyterLab	http://localhost:8888	token <code>anban</code>
Income API	http://localhost:8000	—
Swagger API	http://localhost:8000/docs	—

Variables de entorno (ponlas en cada terminal nueva)

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
export MODEL_URI=models:/heart-failure-clf/Staging
```

Truco: guarda esto en `setenv.sh` y haz `source setenv.sh`.

DVC

```
# Inicializar
dvc init

# Configurar remoto MinIO
dvc remote add -d minio s3://datasets/lab1
dvc remote modify minio endpointurl http://localhost:9000
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345

# Trackear un fichero
dvc add data/raw/heart_failure.csv

# Subir al remoto
dvc push

# Recuperar desde el remoto
dvc pull

# Ejecutar el pipeline
dvc repro

# Ver el grafo del pipeline
dvc dag

# Ver estado
dvc status

# Recuperar un fichero a su versión correcta
dvc checkout data/raw/heart_failure.csv
```

Git + DVC

```
# Commit típico tras dvc add  
git add data/raw/heart_failure.csv.dvc data/raw/.gitignore  
git commit -m "trackea dataset"  
  
# Commit típico tras editar el pipeline  
git add dvc.yaml dvc.lock params.yaml  
git commit -m "ajusta test_size"
```

MLflow

```
# UI
open http://localhost:5000

# Listar experimentos
mlflow experiments search

# Listar runs
mlflow runs list --experiment-id 1
```

Desde Python

```
import mlflow, os
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("heart-failure-classifier")

with mlflow.start_run(run_name="rf"):
    mlflow.log_params({"n_estimators": 300})
    mlflow.log_metrics({"f1": 0.78})
    mlflow.set_tag("owner", "yo")
    mlflow.sklearn.log_model(model, name="model")
```

Model Registry

```
import mlflow
client = mlflow.MlflowClient()

# Registrar
mv = client.create_model_version(
    name="heart-failure-clf",
    source=f"runs://{run_id}/model",
    run_id=run_id,
)

# Promocionar
client.transition_model_version_stage(
    name="heart-failure-clf",
    version=1,
    stage="Staging",
)

# Cargar
m = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")
```

FastAPI + Docker

```
# Local con autoreload
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Build imagen
docker build -t anban/income-api:0.1 -f Dockerfile .

# Run en Linux
docker run --rm -d --network host \
  -e MLFLOW_TRACKING_URI=http://localhost:5000 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:0.1

# Run en Mac/Windows
docker run --rm -d --network anban_default -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:0.1

# Ver logs
docker logs -f income-api

# Stop
docker stop income-api
```

Probar la API con curl

```
# Health
curl -s http://localhost:8000/health | jq

# Version
curl -s http://localhost:8000/version | jq

# Predict (paciente con datos clínicos de insuficiencia cardíaca)
curl -s -X POST http://localhost:8000/predict \
  -H 'content-type: application/json' \
  -d '{
    "age": 75, "anaemia": 0, "creatinine_phosphokinase": 582,
    "diabetes": 0, "ejection_fraction": 20, "high_blood_pressure": 1,
    "platelets": 265000, "serum_creatinine": 1.9, "serum_sodium": 130,
    "sex": 1, "smoking": 0
  }' | jq

# Metrics
curl -s http://localhost:8000/metrics

# Monitor (drift)
curl -s http://localhost:8000/monitor | jq
```

Locust (stress test)

```
locust -f tests/locustfile.py --host http://localhost:8000 \  
      --users 50 --spawn-rate 10 --run-time 30s --headless
```

Evidently y drift

```
# Generar dataset con drift
python synthetic/make_drift.py

# Reporte HTML
python src/drift_report.py \
  --reference ../lab1_dataops/data/processed/test.parquet \
  --current data/processed/drifted.parquet \
  --output reports/drift.html
```

Promote condicional

```
python src/promote_if_better.py \  
  --name heart-failure-clf \  
  --candidate-version 2 \  
  --metric f1 \  
  --min-improvement 0.01
```

Atajos para depuración

```
# Ver puertos ocupados
lsof -i :5000

# Reiniciar un servicio
docker compose -f docker/docker-compose.yml restart minio

# Entrar a un contenedor
docker exec -it anban-mlflow bash

# Ver red de Docker
docker network ls
docker network inspect anban_default
```

Si algo se rompe sin remedio

```
# Reset total del stack (pierdes runs)
docker compose -f docker/docker-compose.yml down -v
docker compose -f docker/docker-compose.yml up -d

# Reset del lab 1
cd labs/lab1_dataops
rm -rf data/processed
dvc repro

# Reentrenar y promocionar de cero
python labs/lab2_mlflow_dvc/src/train.py --model rf
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment heart-failure-classifier --metric f1 --name heart-failure-clf

python labs/lab2_mlflow_dvc/src/promote.py \
    --name heart-failure-clf --version 1 --stage Staging
```